Análise Exploratória de Dados

Asimov Academy

ASIMOV

Conteúdo

01. O que é Análise Exploratória de Dados (EDA)? 4
O que é EDA? . . . 4
Características da EDA . . . 4
A Importância da Prática . . . 4
O Objetivo Final da EDA . . . 5

02. Leitura dos dados 6
Carregando os Dados . . . 6
Explorando os Nomes das Colunas . . . 6
Exemplo de saída: . . . 7
Limpando o Texto com HTML . . . 8
Conclusão . . . 9

03. Entendendo a tabela 10
Explorando a Estrutura com `df.info()` . . . 10
Verificando Dados Nulos . . . 10
Formatando como Percentual . . . 10
Visualizando Graficamente os Dados Faltantes . . . 11

04. Explorando valores numéricos 13
Selecionando Colunas Numéricas com `select_dtypes()` . . . 13
Descrição Estatística com `describe()` . . . 13
Arredondando os Resultados com `round()` . . . 14
Lidando com Colunas Específicas . . . 14
Obtendo Insights . . . 15

05. Explorando dados categóricos 16
Selecionando Dados Categóricos . . . 16
Descrevendo Colunas Categóricas . . . 16
Valores Possíveis com `unique()` . . . 17
Lidando com Múltiplas Respostas . . . 17
Análise Cruzada de Categóricos . . . 17
Conclusão . . . 17

06. Boxplot 18
Preparando o Ambiente . . . 18

Criando o Boxplot . . . 18
Identificando Outliers . . . 19
Cercas de Tukey . . . 19
Personalizando o Boxplot . . . 20
Adicionando Scatterplot . . . 20
Conclusão . . . 21

07. Histograma **22**
Definindo o tamanho dos *bins* . . . 23
Adicionando KDE: estimativa de densidade . . . 24
KDE aplicado a uma distribuição normal . . . 25
Investigando picos inesperados . . . 26
Conclusão . . . 26

08. Comparando histogramas **28**
Preparação dos dados . . . 28
Comparando histogramas por faixa etária . . . 29
Comparando por tipo de trabalho . . . 31
Conclusão . . . 33

09. Violinos **34**
O Gráfico de Violino: Unindo Boxplot e Densidade . . . 35
Enriquecendo com Variáveis Segmentadoras . . . 37
Controlando a Suavização dos Violinos . . . 39

10. Normalidade e outras métricas de dispersão **41**
O que é uma distribuição normal? . . . 41
Parâmetros da curva normal: média e desvio padrão . . . 42
Como avaliar a normalidade dos dados? . . . 43
Avaliação de normalidade em dados reais . . . 44
Transformações para aproximar da normalidade . . . 46

11. Conceito de Correlação **48**
Visualizando Correlações com Gráficos de Dispersão . . . 48
Medindo Correlação com Pearson R . . . 50
Conclusão . . . 52

12. Correlação nos nossos dados **53**
Retomando: o que é Correlação? . . . 53
Por que Filtrar Outliers? . . . 53

Explorando os dados: salário e experiência . . . 53
Lidando com Outliers . . . 54
Segmentando os dados por faixa salarial . . . 55
Calculando a correlação nos dois grupos . . . 56
Visualização com histogramas 2D . . . 56
Correlação por país . . . 57
Conclusão . . . 60

13. Correlação e distribuição de dados **61**
Preparando os Dados . . . 61
Como interpretar . . . 62
Customizando o Pairplot . . . 63
Exemplo com personalização: . . . 63
Jointplot: Foco em Duas Variáveis . . . 64
Exemplo: . . . 65
Versão com customização: . . . 65
Avançado: Adicionando Camadas ao Jointplot . . . 66

14. Cálculo de coeficientes de correlação **69**
Correlação e Linearidade . . . 69
Correlação de Spearman . . . 70
Visualizando Correlações em um DataFrame . . . 70
Gráficos de Dispersão . . . 70
Comparando Pearson e Spearman . . . 71
Quando Usar Pearson ou Spearman? . . . 72
Em termos estatísticos: . . . 72

15. Modelos de regressão linear **73**
O que é regressão linear? . . . 73
Aplicando a regressão linear com `scipy` . . . 73
Visualizando a regressão linear . . . 74
Filtrando os outliers . . . 75
Usando o `seaborn.regplot` . . . 75
Outras possibilidades com `regplot` . . . 76
Considerações finais . . . 76

01. O que é Análise Exploratória de Dados (EDA)?

Nesta apostila, vamos utilizar as bibliotecas **Pandas**, **Matplotlib** e **Seaborn** para explorar conjuntos de dados e visualizar informações por meio de gráficos. Embora o foco não seja o aprofundamento técnico nessas bibliotecas, é recomendável que você já tenha alguma familiaridade com elas a partir de outros cursos da Asimov Academy.

O que é EDA?

A **Análise Exploratória de Dados** é uma etapa fundamental no processo de ciência de dados. Ela envolve o uso de técnicas e ferramentas para examinar um conjunto de dados e extrair informações relevantes e, muitas vezes, surpreendentes.

Em essência, EDA nos ajuda a:

- **Compreender o conteúdo dos dados**: o que está disponível, quais são as variáveis, e como elas se relacionam.
- **Detectar padrões e tendências**: por exemplo, identificar correlações, agrupamentos ou distribuições específicas.
- **Gerar insights valiosos**: muitas vezes, informações escondidas podem ser reveladas ao aplicarmos visualizações e estatísticas básicas.

Características da EDA

A análise exploratória **não segue um roteiro fixo**. Cada projeto pode ter seus próprios desafios, com dados distintos e objetivos variados. Ainda assim, podemos pensar em EDA como uma coleção de passos experimentais, gráficos e metodologias aplicáveis a qualquer novo dataset.

Entre as ações comuns estão:

- Visualizações como histogramas, gráficos de dispersão e boxplots
- Estatísticas descritivas: média, mediana, desvio padrão, etc.
- Agrupamentos e filtragens por categoria
- Análises de correlação

A Importância da Prática

Aqui, utilizaremos um **conjunto de dados padrão**, disponível no material complementar. No entanto, recomendamos fortemente que você **aplique os conceitos aprendidos em seus próprios dados**.

Pode ser um dataset do seu trabalho, algo de interesse pessoal ou até mesmo dados públicos disponíveis na internet. A prática com dados reais é a melhor forma de internalizar o conteúdo e desenvolver uma intuição analítica mais sólida.

O Objetivo Final da EDA

Ao final de uma análise exploratória, buscamos **insights** — informações que antes estavam ocultas e que agora se tornam visíveis graças às ferramentas aplicadas.

Exemplos de possíveis descobertas:

- Uma categoria de clientes que gera maior receita
- Um comportamento atípico nos dados de produção
- Um padrão de crescimento em vendas ao longo do tempo

Esses achados geralmente exigem **experimentação**. É preciso testar diferentes visualizações, aplicar filtros, agrupar dados e ir ajustando o olhar até que algo relevante surja.

02. Leitura dos dados

Neste curso, utilizaremos os dados da **pesquisa anual do Stack Overflow de 2022**, que reúne informações sobre o perfil dos profissionais de tecnologia: suas carreiras, formas de aprendizado, linguagens utilizadas e outras curiosidades.

A pesquisa é realizada com usuários da plataforma Stack Overflow — um site amplamente conhecido por sua comunidade de perguntas e respostas voltadas à programação.

Além dos dados brutos, o próprio site apresenta análises prévias sobre aspectos como:

- Nível de escolaridade dos participantes
- Métodos de aprendizado (online, livros, escola, amigos)
- Linguagens mais utilizadas ou desejadas

Esses dados estão disponíveis para download e já fazem parte do material do curso.

Carregando os Dados

Começamos importando o `pandas` e lendo o arquivo principal da pesquisa:

```
import pandas as pd
```

```
df = pd.read_csv("materiais_de_aula/survey_results_public.csv", index_col="ResponseId")
```

A opção `index_col="ResponseId"` garante que a coluna de ID de resposta seja usada como índice, evitando duplicação com a numeração padrão do DataFrame.

Ao carregar o DataFrame, temos:

- **73 mil linhas** (respostas)
- **79 colunas**, cada uma representando uma pergunta do questionário

Explorando os Nomes das Colunas

Podemos verificar os nomes das colunas com:

```
df.columns
```

Cada coluna representa uma pergunta feita na pesquisa, como por exemplo:

- `LanguageWantToWorkWith`: linguagens que os participantes desejam trabalhar
- `CompTotal`: compensação total (salário)

Algumas colunas são compreensíveis pelo nome, mas outras exigem uma explicação mais detalhada.
Lendo o Esquema das Perguntas

O Stack Overflow também disponibiliza um segundo arquivo, chamado `survey_results_schema.csv`, que contém a descrição de cada coluna:

```
perguntas = pd.read_csv("materiais_de_aula/survey_results_schema.csv", index_col="qid")
```

Este arquivo inclui:

- O nome da coluna (`Column`)
- O texto da pergunta original (`Question`)

Podemos exibir os pares nome–descrição com:

```
for nome, descricao in zip(schema['Column'], schema['Question']):
    print(f"{nome} → {descricao}")
```

Exemplo de saída:

```
<p><span style="font-size:16px; font-family:arial,helvetica,sans-serif;">The next set of questio
<br>
Most questions in this section are optional. Required questions are noted with *.</span></p>
Language → Which <b>programming, scripting, and markup languages</b> have you done extensive dev
Database → Which <b>database environments </b>have you done extensive development work in over t
Platform → Which <b>cloud platforms</b> have you done extensive development work in over the pas
Webframe → Which <b>web frameworks and web technologies</b> have you done extensive development
MiscTech → Which <b>other frameworks and libraries</b> have you done extensive development work
ToolsTech → Which <b>developer </b><strong>tools</strong> have you done extensive development wo
NEWCollabTools → Which <strong>development environments</strong> did you use regularly over the
OpSys → What is the primary <b>operating system</b> in which you work? *
VersionControlSystem → What are the primary <b>version control systems</b> you use? Select all t
VCInteraction → How do you interact with your version control system? Select all that apply.
VCHosting → What version control hosting service are you using?
OfficeStackAsync → Which <b>collaborative work management tools</b> did you use regularly over t
OfficeStackSync → Which <b>communication tools</b> did you use regularly over the past year, and
Blockchain → How favorable are you about blockchain, crypto, and decentralization?
S4 → <span style="font-size:22px; font-family: arial,helvetica,sans-serif; font-weight: 700;">St
<br>

<p><span style="font-size:16px; font-family:arial,helvetica,sans-serif;">This section will focus
<br>
Most questions in this section are optional. Required questions are noted with *.</span></p>
NEWSOSites → Which of the following Stack Overflow sites have you visited? Select all that apply
SOVisitFreq → How frequently would you say you visit Stack Overflow?
SOAccount → Do you have a Stack Overflow account?
SOPartFreq → How frequently would you say you participate in Q&amp;amp;A on Stack Overflow? By parti
SOComm → Do you consider yourself a member of the Stack Overflow community?
```

Figure 1: Exemplo de saída

Limpando o Texto com HTML

Algumas perguntas vêm com tags HTML incorporadas. Para facilitar a leitura, podemos remover esses elementos com uma função auxiliar:

```
import re

def remove_html(texto):
    padrao = re.compile(r"<.*?>")
    return padrao.sub("", texto)
```

Aplicando a limpeza nas perguntas:

```
for nome, descricao in zip(perguntas['qname'], perguntas['question']):
    print(f"{nome} → {remove_html(descricao)}")
```

Isso resulta em uma visualização mais clara dos enunciados das perguntas.

```
The next set of questions will focus on technology and tech culture.

Most questions in this section are optional. Required questions are noted with *.
Language → Which programming, scripting, and markup languages have you done extensive developmen
Database → Which database environments have you done extensive development work in over the past
Platform → Which cloud platforms have you done extensive development work in over the past year,
Webframe → Which web frameworks and web technologies have you done extensive development work in
MiscTech → Which other frameworks and libraries have you done extensive development work in over
ToolsTech → Which developer tools have you done extensive development work in over the past year
NEWCollabTools → Which development environments did you use regularly over the past year, and wh
OpSys → What is the primary operating system in which you work? *
VersionControlSystem → What are the primary version control systems you use? Select all that app
VCInteraction → How do you interact with your version control system? Select all that apply.
VCHosting → What version control hosting service are you using?
OfficeStackAsync → Which collaborative work management tools did you use regularly over the past
OfficeStackSync → Which communication tools did you use regularly over the past year, and which
Blockchain → How favorable are you about blockchain, crypto, and decentralization?
S4 → Stack Overflow - usage and community


This section will focus on Stack Overflow usage and community questions.

Most questions in this section are optional. Required questions are noted with *.
NEWSOSites → Which of the following Stack Overflow sites have you visited? Select all that apply
SOVisitFreq → How frequently would you say you visit Stack Overflow?
SOAccount → Do you have a Stack Overflow account?
SOPartFreq → How frequently would you say you participate in Q&amp;amp;A on Stack Overflow? By parti
SOComm → Do you consider yourself a member of the Stack Overflow community?
S5 → Demographics


All questions in this section are optional, you may leave questions blank if you prefer not to a
Age → What is your age?
Gender → Which of the following describe you, if any? Please check all that apply.
Trans → Do you identify as transgender?
Sexuality → Which of the following describe you, if any? Please check all that apply.
Ethnicity → Which of the following describe you, if any? Please check all that apply.
```

Figure 2: Exemplo de saída já com a limpeza aplicada

Conclusão

Com esses dois arquivos — o de dados e o das perguntas — conseguimos:

- **Carregar e visualizar os dados**
- **Entender o significado das colunas**
- **Preparar o ambiente para análises futuras**

Esse entendimento inicial é essencial para que as próximas etapas da EDA façam sentido, pois será com base nessas perguntas que exploraremos o comportamento e os padrões presentes nos dados.

03. Entendendo a tabela

Agora que já entendemos o contexto dos dados, de onde eles vêm e o que representam, é hora de explorar mais profundamente o conteúdo da tabela utilizando métodos do `pandas`.

Explorando a Estrutura com `df.info()`

O método `df.info()` fornece um resumo geral da tabela:

```
import pandas as pd

df = pd.read_csv("dados.csv")
df.info()
```

Esse método mostra:

- O número total de entradas (linhas);
- O nome de cada coluna;
- O número de valores não nulos em cada coluna;
- O tipo de dado de cada coluna (`object`, `float64`, `int64`, etc).

Em muitos casos, especialmente com dados importados de formulários, a maioria das colunas será do tipo `object`, que geralmente representa strings. Esses valores são mais difíceis de trabalhar em análises estatísticas, já que não podemos fazer médias ou somas diretamente. Por outro lado, colunas numéricas, como `CompTotal`, aparecem como `float64` e são mais adequadas para esse tipo de análise.

Verificando Dados Nulos

Podemos usar `df.isnull()` para preencher o DataFrame com valores True/False, de acordo com o valor em cada célula ser nulo ou não:

```
df.isnull()
```

Como `True` é equivalente ao número `1` em Python, podemos simplesmente usar o método `df.sum()` para encontrar o número de valores nulos em cada coluna:

```
df.isnull().sum()
```

Formatando como Percentual

```
df.isna().sum() / len(df)
```

E finalmente, podemos ordenar e adicionar um pouco de formatação ao output.

Fazemos isso definindo a nossa própria função de formatação de valores percentuais, e depois a entregamos para o método `apply()`:

```
def format_percent(value: float) -> str:

    """Retorna o float formatado como valor percentual."""

    return f'{100 * value:.2f}%'
(df.isna().sum() / len(df)).sort_values(ascending=False).apply(format_percent)
```

Visualizando Graficamente os Dados Faltantes

Para uma visualização mais intuitiva, podemos plotar a porcentagem de dados nulos com `seaborn` e `matplotlib`:

```
import matplotlib.pyplot as plt
import seaborn as sns

percent_data = (df.isna().sum() / len(df)).sort_values(ascending=False).reset_index()
percent_data.columns = ['column_name', 'percent_na']

fig, ax = plt.subplots(figsize=(18, 8))


sns.barplot(data=percent_data, ax=ax, x='column_name', y='percent_na')
fig.suptitle('% de dados faltantes em cada coluna')
plt.xticks(rotation=80)

plt.show()
```

Abaixo, um exemplo do gráfico gerado pelo código acima:

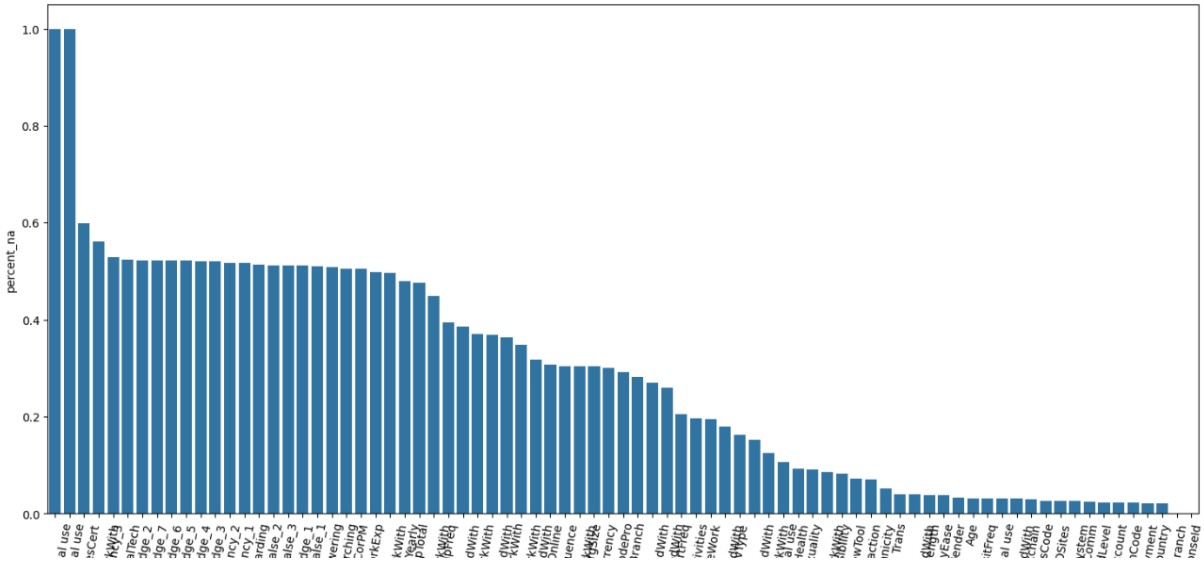


Figure 3: Exibição através de um gráfico simples

Esse gráfico ajuda a visualizar quais colunas têm muitos dados ausentes. No exemplo analisado, duas colunas tinham 100% de valores nulos (completamente inúteis), e várias outras com mais de 50%.

Essa visualização é útil para qualquer conjunto de dados e pode ser reutilizada em outros projetos de análise.

No próximo capítulo, vamos explorar os diferentes tipos de dados existentes nas colunas e como podemos manipulá-los para facilitar a análise.

04. Explorando valores numéricos

Agora que já entendemos melhor a estrutura da nossa tabela, vamos focar nas colunas numéricas.

Selecionando Colunas Numéricas com `select_dtypes()`

Para selecionar colunas de um tipo específico, como `float`, usamos:

```
df.select_dtypes(include='float')
```

Esse comando retorna apenas as colunas cujo tipo de dado é `float`. Caso queira selecionar apenas inteiros:

```
df.select_dtypes(include='int')
```

Se nenhuma coluna for do tipo `int`, o retorno será um DataFrame vazio. Para selecionar todos os tipos numéricos (tanto `int` quanto `float`), usamos:

```
df.select_dtypes(include='number')
```

Essa abordagem é útil para separar colunas numéricas de outras categorias que não podem ser usadas em operações como média ou soma.

Descrição Estatística com `describe()`

O método `df.describe()` gera um resumo estatístico das colunas numéricas:

```
df.describe()
```

Esse método retorna:

```
df.describe()
```

	CompTotal	VCHostingPersonal use	VCHostingProfessional use	WorkExp	ConvertedCompYearly
count	3.842200e+04	0.0	0.0	36769.000000	3.807100e+04
mean	2.342434e+52	NaN	NaN	10.242378	1.707613e+05
std	4.591478e+54	NaN	NaN	8.706850	7.814132e+05
min	0.000000e+00	NaN	NaN	0.000000	1.000000e+00
25%	3.000000e+04	NaN	NaN	4.000000	3.583200e+04
50%	7.750000e+04	NaN	NaN	8.000000	6.784500e+04
75%	1.540000e+05	NaN	NaN	15.000000	1.200000e+05
max	9.000000e+56	NaN	NaN	50.000000	5.000000e+07

Figure 4: Exemplo do resultado do df.describe

Podemos ver que os valores nas colunas não são muito informativos.

Como vimos anteriormente, as colunas `VCHostingPersonal use` e `VCHostingProfessional use` possuem apenas valores nulos. Além disso, `CompTotal` é apenas a compensação (salário) em dinheiro local: o valor convertido dólares está em `ConvertedCompYearly`.

Podemos selecionar apenas as colunas de interesse, antes de chamar o método `df.describe()`:

```
df[['WorkExp', 'ConvertedCompYearly']].describe()
```

	WorkExp	ConvertedCompYearly
count	36769.000000	3.807100e+04
mean	10.242378	1.707613e+05
std	8.706850	7.814132e+05
min	0.000000	1.000000e+00
25%	4.000000	3.583200e+04
50%	8.000000	6.784500e+04
75%	15.000000	1.200000e+05
max	50.000000	5.000000e+07

Figure 5: Colunas de interesse com o df.describe

Arredondando os Resultados com `round()`

Para tornar os números mais legíveis, podemos arredondar os valores:

```
df[['WorkExp', 'ConvertedCompYearly']].describe().round(2)
```

Lidando com Colunas Específicas

Você também pode aplicar o `describe()` em colunas específicas:

```
df[['WorkExp', 'ConvertedCompYearly']].describe()
```

> Observação: a coluna `CompTotal`, apesar de ser numérica, pode conter salários em diferentes moedas ou períodos de tempo, tornando-a inconsistente. Por isso, preferimos utilizar a coluna

> `ConvertedCompYearly`, que já foi padronizada para dólar anual.

Obtendo Insights

Mesmo sem gráficos, conseguimos extrair insights valiosos a partir dos dados numéricos:

- A média de anos de experiência (`WorkExp`) é de aproximadamente 10,2 anos.
- A média salarial anual (`ConvertedCompYearly`) é de cerca de 170 mil dólares.

Esses valores podem estar distorcidos por outliers, o que percebemos ao comparar a média com a mediana (`50%`). Se a mediana for menor que a média, é sinal de que há valores extremos puxando a média para cima.

Além disso, o método `describe()` ignora automaticamente os valores nulos, o que evita erros ao calcular métricas.

Na próxima aula, vamos explorar como lidar com colunas categóricas e de texto.

05. Explorando dados categóricos

Agora vamos explorar como trabalhar com dados que não são numéricos — os chamados dados categóricos. Eles geralmente representam categorias, como tipos de trabalho, faixas etárias ou preferências tecnológicas.

Selecionando Dados Categóricos

Muita gente pensa em usar `select_dtypes(include='string')`, mas esse comando não retorna nenhuma coluna, pois a maioria dos dados categóricos vem como `object`. O correto é:

```
df.select_dtypes('object')
```

Se o DataFrame original tem 78 colunas e esse retorno tem 73, significa que 5 colunas eram numéricas.

> **Nota**: `object` em Pandas é um tipo genérico que pode conter qualquer tipo de dado. Com o Pandas 2.0 (2023), foi introduzido o tipo `string`, mas por compatibilidade, `object` ainda é padrão.

Para forçar a conversão de uma coluna para o tipo `string`:

```
df['RemoteWork'].astype('string')
```

Se houver valores nulos, como None, eles serão convertidos para NA, um nulo do tipo texto.

Por enquanto seguiremos com colunas `object`, mas é bom saber da existência do tipo de dado `string` caso você precise garantir que todos os elementos da coluna são texto!

Descrevendo Colunas Categóricas

Para colunas categóricas, usamos `value_counts()` para ver a frequência dos valores:

```
df['RemoteWork'].value_counts()
```

Adicionando parâmetros úteis:

```
df['RemoteWork'].value_counts(normalize=True, dropna=False)
```

- `normalize=True`: mostra proporções
- `dropna=False`: inclui valores nulos

Isso nos permite entender, por exemplo, que 19% dos participantes não responderam à pergunta.

Valores Possíveis com `unique()`

Para obter todos os valores distintos de uma coluna:

```
df['RemoteWork'].unique()
```

Nem sempre os valores são bem definidos. A coluna `LanguageHaveWorkedWith` é um exemplo:

```
df['LanguageHaveWorkedWith'].unique()
```

Essa coluna tem cerca de 25 mil valores únicos, pois permite múltiplas escolhas (separadas por ponto e vírgula).

Lidando com Múltiplas Respostas

É possível expandir essas respostas para contar individualmente:

```
langs = df['LanguageHaveWorkedWith'].dropna().str.split(';')
langs_exploded = langs.explode()
langs_exploded.value_counts()
```

Assim conseguimos ver a popularidade de cada linguagem.

Análise Cruzada de Categóricos

Também podemos cruzar colunas categóricas:

```
df[['Age', 'RemoteWork']].value_counts().sort_index()
```

Isso mostra quantas pessoas de cada idade responderam cada categoria de trabalho remoto. Quanto mais colunas, mais combinações — e mais difícil interpretar. Mas ainda pode ser útil para encontrar padrões.

Conclusão

Valores categóricos não se analisam com médias, mas sim com contagens e proporções. Com eles, podemos entender a distribuição de preferências, comportamentos e segmentações importantes para a análise exploratória.

06. Boxplot

Agora, vamos analisar os dados de uma forma mais visual, começando com um gráfico de **boxplot**. O boxplot é uma ferramenta poderosa para visualizar a distribuição dos dados, mostrando a posição dos quartis e identificando valores atípicos (outliers).

Preparando o Ambiente

Importamos as bibliotecas necessárias para o trabalho:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ajustando o estilo do seaborn
sns.set()
sns.set_context('poster')
```

Agora, vamos carregar o dataset e observar a distribuição dos dados na coluna `WorkExp` (anos de experiência no trabalho):

```
df = pd.read_csv('materiais de aula/survey_results_public.csv', index_col='ResponseId')

df['WorkExp'].describe().round(2)
```

O método `describe()` nos mostra os principais estatísticos, como a média, desvio padrão e percentis. ## O que é um Boxplot?

O **boxplot** é um gráfico que exibe a distribuição básica de um conjunto de valores. Ele mostra os seguintes elementos:

- **Q1 (25%)**: O primeiro quartil, o valor abaixo do qual 25% dos dados se encontram.
- **Q2 (Mediana, 50%)**: O valor central da distribuição.
- **Q3 (75%)**: O terceiro quartil, o valor abaixo do qual 75% dos dados se encontram.

Além disso, o boxplot marca os **outliers**, que são valores muito distantes dos quartis.

Criando o Boxplot

Vamos agora plotar um **boxplot** para visualizar os dados de anos de experiência com a função `sns.boxplot()`:

```
fig, ax = plt.subplots(figsize=(22, 8))
sns.boxplot(data=df, ax=ax, x='WorkExp')
fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 6: Gráfico boxplot plotado

O gráfico gerado apresenta a distribuição de dados e os quartis, conforme esperado. Por exemplo, o quartil 25% (Q1) é 4, o Q2 (mediana) é 8, e o Q3 é 15. Isso é exatamente o que vimos com a função `describe()`.

Identificando Outliers

No boxplot, podemos ver que há alguns **outliers**, que são valores extremos em relação à distribuição. O seaborn usa o método de **cercas de Tukey** para definir os outliers.

Cercas de Tukey

A detecção de outliers é feita com base na **distância interquartil (IQR)**, que é a diferença entre o quartil 75% (Q3) e o quartil 25% (Q1). Os valores que estão além de `1.5 * IQR` a partir de Q3 ou Q1 são considerados outliers.

Podemos ajustar o valor de multiplicação de `1.5` para outro valor, usando o parâmetro `whis`. Vamos modificar o código para usar `whis=3.0`:

Figure 7: Gráfico boxplot plotado

Agora, o limiar para a definição de outliers foi ajustado. Com `whis=3.0`, os valores mais extremos são considerados outliers.

Que valor de `whis` utilizar? Vai depender do seu entendimento e conhecimento dos seus dados.

Personalizando o Boxplot

Existem outros métodos de determinação de outliers - na realidade, o estudo de detecção de outliers é todo um campo dentro da estatística. Isto vai além do escopo deste curso, mas é importante saber que as cercas de Tukey nem sempre são adequadas para isso. Na realidade, as cercas costumam funcionar melhor em dados simétricos, o que não é o caso da coluna `WorkExp`!

Embora o boxplot mostre as distribuições de dados, ele não revela a posição de cada valor individual. Para visualizar isso, podemos adicionar um gráfico de dispersão (`scatterplot`) sobre o boxplot, utilizando a função `sns.scatterplot()`.

Adicionando Scatterplot

Primeiro, geramos um vetor de **ruído** para os dados no eixo X, para que os pontos não se sobreponham. Em seguida, plotamos os dados individuais no gráfico:

```
import numpy as np
# Gerando ruído no eixo X
x_noise = np.random.uniform(-0.1, 0.1, len(df))
```

```
x_scatter = df['WorkExp'] + x_noise
# Gerando valores aleatórios para o eixo Y
y_scatter = np.random.uniform(-0.3, 0.3, len(df))
# Plotando o boxplot e o scatterplot
fig, ax = plt.subplots(figsize=(22, 8))
sns.boxplot(data=df, ax=ax, x='WorkExp', whis=3.0)
sns.scatterplot(data=df, ax=ax, x=x_scatter, y=y_scatter, alpha=0.8, s=20)
fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 8: Gráfico boxplot plotado

Neste gráfico, os dados individuais são representados por pontos sobre o boxplot. Agora podemos ver com mais clareza como os dados estão distribuídos, especialmente nos valores mais baixos de anos de experiência.

Conclusão

O **boxplot** é uma ótima ferramenta para analisar a distribuição de dados e identificar outliers. Embora o método de **cercas de Tukey** seja amplamente utilizado, a escolha do valor para o parâmetro `whis` e a interpretação dos outliers dependem do seu conhecimento sobre os dados.

07. Histograma

O histograma é um tipo de gráfico que agrupa valores em "caçapas" (*bins*) e representa sua distribuição por meio de barras verticais.

- Ele ajuda a visualizar a distribuição dos dados numéricos.
- Cada barra representa a quantidade de elementos que caem dentro de um intervalo (bin).

Vale lembrar que , antes de executar o código da geração do gráfico, assim como no capítulo anterior, é importante fazer as importações e definir o DataFrame:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carregamento dos dados
df = pd.read_csv('materiais de aula/survey_results_public.csv', index_col='ResponseId')

# Ajuste de estilo dos gráficos
sns.set()
sns.set_context('poster')
```

Vamos criar um histograma para a coluna `WorkExp`, que representa os anos de experiência dos participantes:

```
fig, ax = plt.subplots(figsize=(12, 8))
sns.histplot(data=df, ax=ax, x='WorkExp')
fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 9: Gráfico histograma plotado

Observa-se uma maior concentração nos primeiros anos (0 a 10), com a frequência caindo conforme os anos aumentam.

Definindo o tamanho dos *bins*

Por padrão, o Seaborn escolhe automaticamente os cortes dos *bins*. Isso pode dificultar a interpretação.

Como sabemos que os dados são inteiros (anos completos), faz sentido definir a largura do bin como 1:

```
fig, ax = plt.subplots(figsize=(12, 8))
sns.histplot(data=df, ax=ax, x='WorkExp', binwidth=1.0)
fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 10: Gráfico histograma plotado

Agora, cada barra representa exatamente um ano de experiência, facilitando a leitura e interpretação.

Adicionando KDE: estimativa de densidade

Podemos complementar o histograma com uma curva suavizada chamada KDE (*Kernel Density Estimate*):

```
fig, ax = plt.subplots(figsize=(12, 8))
sns.histplot(data=df, ax=ax, x='WorkExp', binwidth=1.0, kde=True)
fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 11: Gráfico histograma plotado

O KDE estima uma "curva" para representar a densidade dos dados em cada região. Mesmo sem uma distribuição matemática definida, ele nos ajuda a visualizar o formato da distribuição.

KDE aplicado a uma distribuição normal

Para esse exemplo, vamos usar o `np.random.normal`, onde os dados estão sendo gerados artificialmente usando a função `np.random.normal()` do NumPy. Essa função cria uma amostra de dados aleatórios que seguem uma distribuição normal padrão (média 0 e desvio padrão 1, por padrão). Isso é útil para demonstrar como o KDE consegue capturar o **formato idealizado de uma distribuição** quando os dados têm uma forma conhecida.

```
import numpy as np
valores = np.random.normal(size=10_000)

fig, ax = plt.subplots(figsize=(12, 8))
sns.histplot(valores, kde=True)
plt.title('Distribuição Normal com KDE')
plt.show()
```

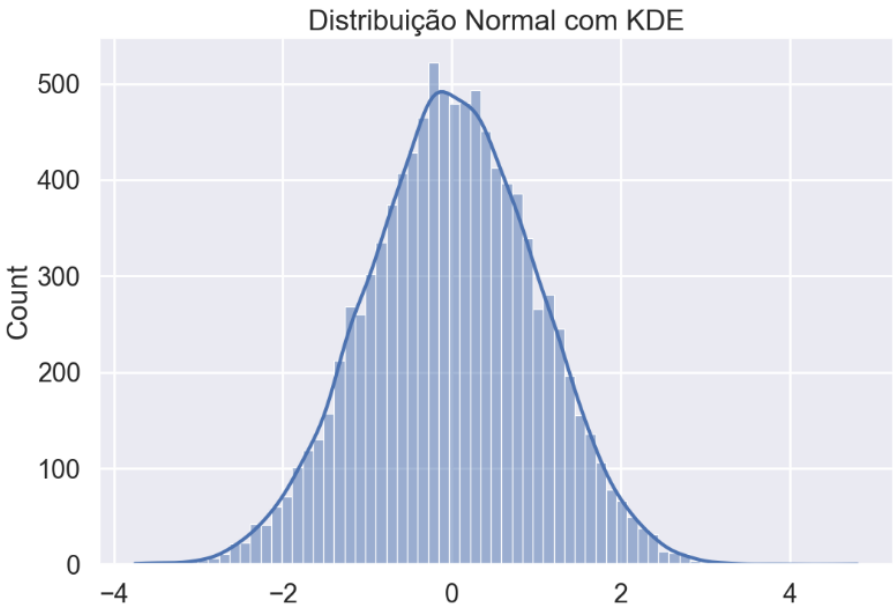


Figure 12: Gráfico histograma plotado

Essa curva suave em forma de sino é o **formato clássico** da distribuição normal. Ao aplicar o KDE, conseguimos visualizá-lo mesmo quando os dados são amostrados de forma aleatória.

Investigando picos inesperados

O KDE mostra uma tendência de queda na distribuição, mas também revela alguns picos em pontos específicos.

Vamos investigar com `value_counts()`:

```
df['WorkExp'].value_counts().sort_values(ascending=False)
```

Notamos que valores como 5, 10, 15, 20, 25 aparecem com frequência alta. Provável motivo: **Arredondamento nas respostas**. As pessoas tendem a arredondar para múltiplos de 5 ou outros números "convenientes", como 12, 15, 20. Isso introduz um viés nos dados.

Conclusão

O histograma é uma ferramenta poderosa na Análise Exploratória de Dados (EDA), pois:

- Facilita a visualização da distribuição dos dados.
- Permite detectar comportamentos inusitados (como o arredondamento).
- Ajuda a gerar **insights** que não seriam evidentes em tabelas.

EDA é sobre experimentar, visualizar e pensar criticamente sobre os dados!

08. Comparando histogramas

Em muitos casos, queremos entender nossos dados comparando diferentes grupos. Para isso, histogramas com segmentação por categorias são muito úteis.

Vamos explorar como fazer isso utilizando a coluna Age como exemplo.

Preparação dos dados

Antes de mais nada, é necessário fazer as importações e inicializar o nosso DataFrame, assim como nas aulas anteriores:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carregamento dos dados
df = pd.read_csv('materiais de aula/survey_results_public.csv', index_col='ResponseId')

# Ajuste de estilo dos gráficos
sns.set()
sns.set_context('notebook')
```

Primeiro, observamos que a coluna Age não traz idades numéricas, mas sim faixas etárias como strings. Isso é útil: são poucas categorias e fáceis de interpretar. Vamos conferir quais são:

```
df['Age'].unique()
```

Agora, vamos avaliar a distribuição de respostas:

```
df['Age'].value_counts(normalize=True, dropna=False).apply(lambda p: f'{100 * p:.2f}%')
```

As categorias “Prefer not to say” e valores nulos somam menos de 4% — podemos ignorá-las sem grandes perdas.

Vamos padronizar os rótulos de faixa etária com intervalos numéricos:

```
import numpy as np

mapa_idades = {

    'Under 18 years old': '[00 - 18]',
    '18-24 years old':    '[18 - 24]',
    '25-34 years old':    '[25 - 34]',
    '35-44 years old':    '[35 - 44]',
    '45-54 years old':    '[45 - 54]',
    '55-64 years old':    '[55 - 64]',
    '65 years or older':  '[65 - 99]',
    'Prefer not to say':  '[?? - ??]',
```

```
    np.nan:                '[?? - ??]',
}


df['AgeRange'] = df['Age'].map(mapa_idades)
df['AgeRange'].value_counts(dropna=False)  # confirmando que os nomes foram modificados
```

Agora, filtramos os dados desconhecidos:

```
df2 = df.loc[df['AgeRange'] != '[?? - ??]']
```

Comparando histogramas por faixa etária

Vamos observar como os anos de experiência variam conforme a faixa etária. Primeiro, com histogramas em contagem absoluta:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
sns.set_context('notebook')

fig, ax = plt.subplots(figsize=(12, 8))

idades = sorted(df2['AgeRange'].unique())

sns.histplot(
    data=df2,
    ax=ax,
    x='WorkExp',
    hue='AgeRange',
    binwidth=1,
    kde=True,
    hue_order=idades
)

fig.suptitle('Anos de experiência programando')
plt.show()
```

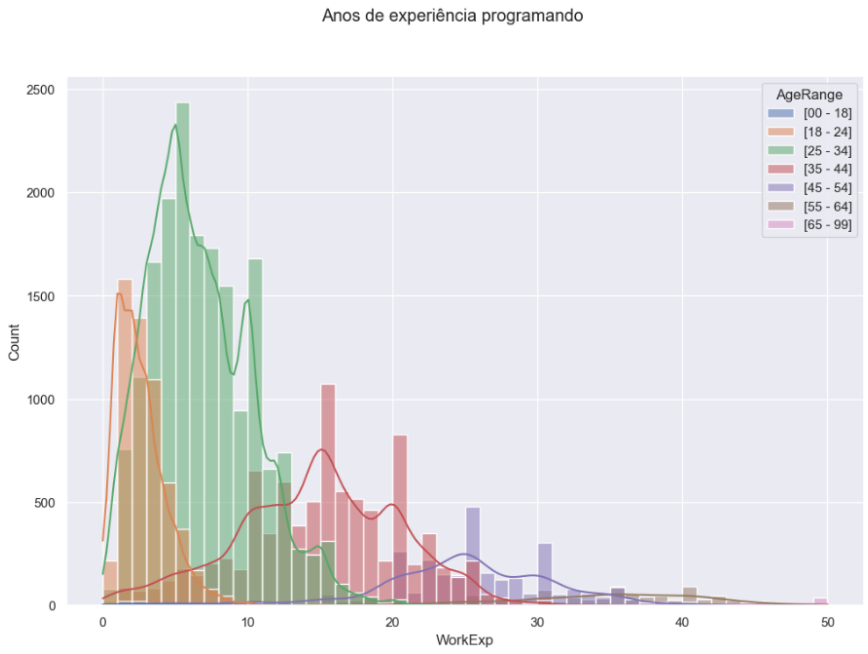


Figure 13: Gráfico histograma plotado

A distribuição faz sentido: pessoas mais jovens têm menos experiência, enquanto as mais velhas têm mais tempo no mercado.

No entanto, como os grupos têm tamanhos diferentes, os histogramas não são diretamente comparáveis. Vamos normalizar:

```
fig, ax = plt.subplots(figsize=(12, 8))

sns.histplot(
    data=df2,
    ax=ax,
    x='WorkExp',
    hue='AgeRange',
    binwidth=1,
    kde=True,
    hue_order=idades,
    stat='density',
    common_norm=False
)

fig.suptitle('Anos de experiência programando (distribuição relativa)')
plt.show()
```

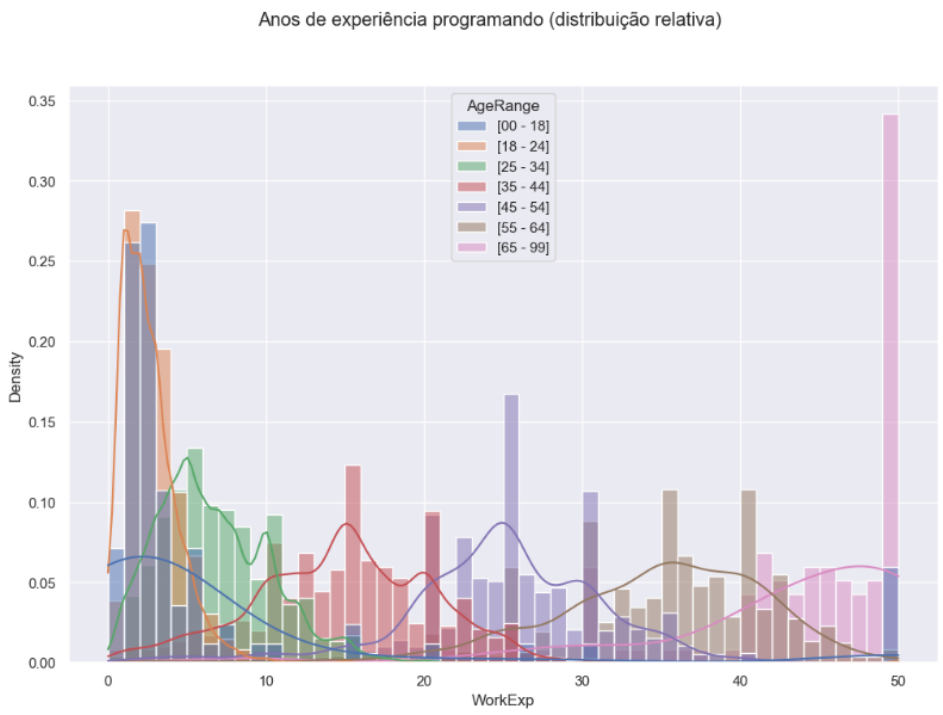


Figure 14: Gráfico histograma plotado

Agora conseguimos comparar as distribuições relativas entre os grupos, independentemente do tamanho.

Comparando por tipo de trabalho

Outra variável interessante é `RemoteWork`, que indica se a pessoa trabalha de forma remota, híbrida ou presencial:

```
df['RemoteWork'].value_counts(normalize=True, dropna=False).apply(lambda p: f'{100 * p:.2f}%')
```

Vamos plotar as distribuições sem os valores nulos:

```
fig, ax = plt.subplots(figsize=(12, 8))

sns.histplot(
    data=df,
    ax=ax,
    x='WorkExp',
    hue='RemoteWork',
    binwidth=1,
    kde=True
)
```

```
fig.suptitle('Anos de experiência programando por tipo de trabalho')
plt.show()
```

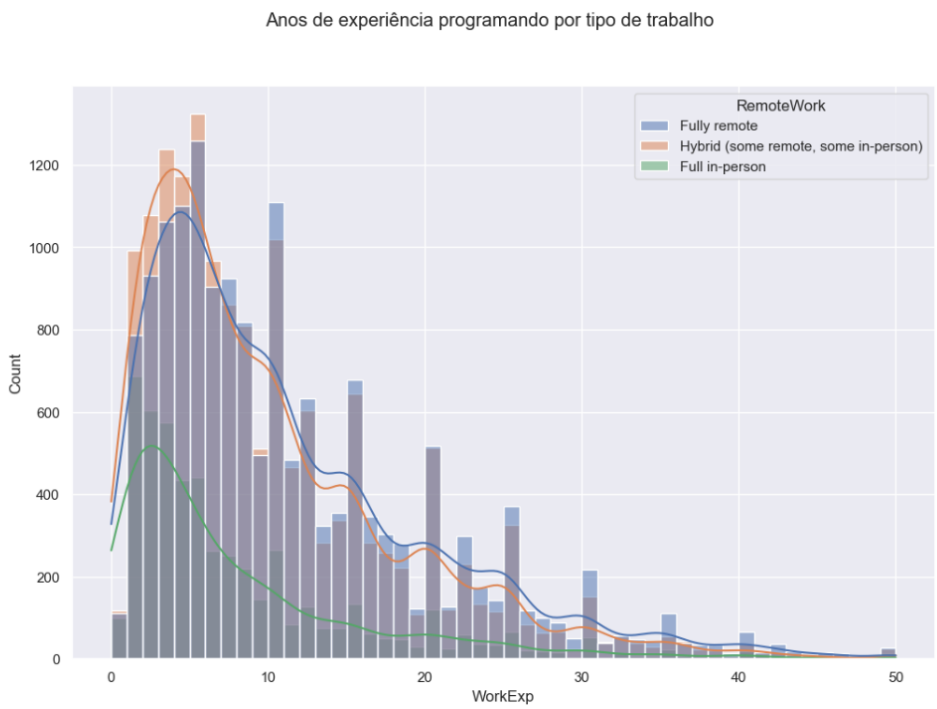


Figure 15: Gráfico histograma plotado

E agora, com a normalização:

```
fig, ax = plt.subplots(figsize=(12, 8))

sns.histplot(
    data=df,
    ax=ax,
    x='WorkExp',
    hue='RemoteWork',
    binwidth=1,
    kde=True,
    stat='density',
    common_norm=False
)

fig.suptitle('Anos de experiência programando (distribuição relativa por tipo de trabalho)')
plt.show()
```

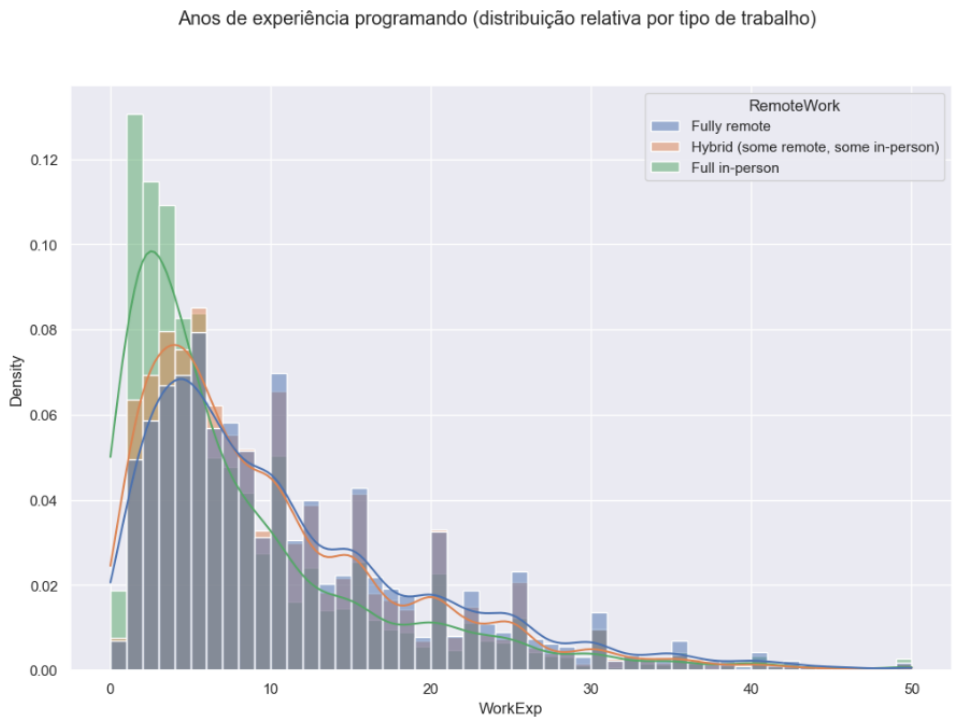


Figure 16: Gráfico histograma plotado

Conclusão

Neste capítulo, exploramos como comparar distribuições de diferentes grupos por meio de histogramas utilizando a biblioteca Seaborn. Inicialmente, revisamos a função `sns.histplot()` e aprendemos a aplicar o parâmetro `hue` para distinguir as categorias de uma variável dentro do mesmo gráfico. Também destacamos a importância de configurar corretamente os parâmetros `bins` (número de intervalos) e `multiple` (forma de sobreposição ou separação dos histogramas), enfatizando como essas escolhas afetam a legibilidade e interpretação do gráfico. Além disso, discutimos como o uso de `kde=True` pode complementar a visualização ao adicionar uma curva suavizada de densidade, útil para observar tendências gerais da distribuição. Finalizamos reforçando a utilidade dessa técnica para comparar grupos dentro de um dataset e identificar padrões ou diferenças significativas em suas distribuições, o que pode ser crucial para análises exploratórias de dados.

09. Violinos

Antes de explorarmos o gráfico de violino, é essencial entender os dois elementos principais que o compõem: o boxplot e o histograma.

O **boxplot** é uma representação gráfica que resume uma distribuição de dados com base nos quartis: o primeiro (Q1), a mediana (Q2) e o terceiro (Q3). Além disso, ele destaca valores extremos (outliers), ajudando a identificar desvios importantes na amostra. Já o **histograma** mostra como os dados estão distribuídos ao longo de intervalos (bins), revelando, por exemplo, se os dados são simétricos, assimétricos ou apresentam caudas longas.

Podemos ilustrar essa diferença com um gráfico duplo, combinando histograma com densidade e boxplot:

```
quantile_args = {
    'linewidth': 2,
    'color': '#453698',
    'linestyle': '--',
}

mean_args = {

    'linewidth': 1,
    'color': '#342345',
    'linestyle': '-',
}



fig, (ax_cima, ax_baixo) = plt.subplots(figsize=(12, 8), nrows=2, sharex=True)
sns.histplot(data=df, ax=ax_cima, x='WorkExp', binwidth=1, kde=True)

for q in [0.25, 0.5, 0.75]:
    ax_cima.axvline(df['WorkExp'].quantile(q), **quantile_args)
ax_cima.axvline(df['WorkExp'].mean(), **mean_args)



sns.boxplot(data=df, ax=ax_baixo, x='WorkExp', whis=3.0)
ax_baixo.axvline(df['WorkExp'].mean(), **mean_args)



fig.suptitle('Anos de experiência programando')


plt.show()
```

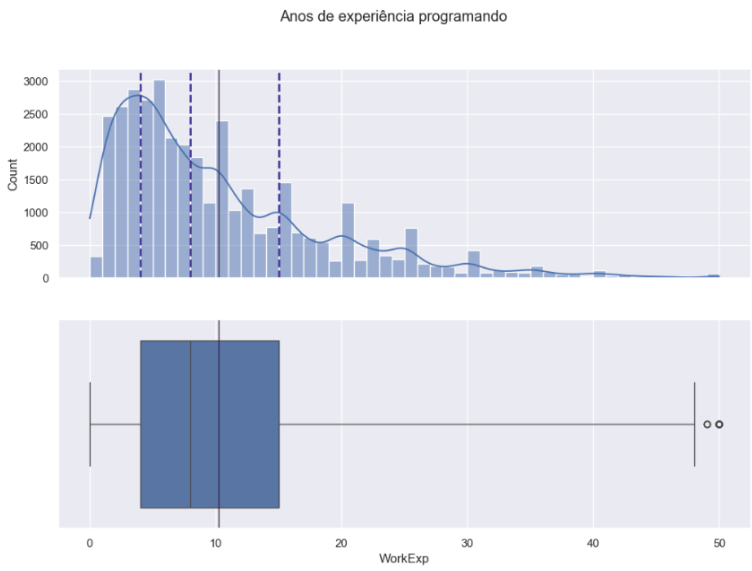


Figure 17: Gráfico violino plotado

Nesse exemplo, a parte superior mostra um histograma com curva de densidade (KDE) da variável `workexp`, além de linhas verticais que indicam os quartis e a média. A parte inferior exibe o boxplot da mesma variável. Juntos, os dois gráficos nos ajudam a compreender não só a concentração dos dados, mas também a sua distribuição e presença de outliers.

O Gráfico de Violino: Unindo Boxplot e Densidade

O **violin plot**, ou gráfico de violino, é uma representação gráfica que combina as vantagens do boxplot com a curva de densidade dos dados. Ele nos permite observar tanto os valores centrais (como mediana e quartis) quanto a distribuição completa dos dados ao longo do eixo.

Veja abaixo um exemplo básico com a variável `workexp`:

```
quantile_args = {
    'linewidth': 2,
    'color': '#453698',
    'linestyle': '--',
}

mean_args = {
    'linewidth': 1,
    'color': '#342345',
    'linestyle': '-',
}
```

```
fig, ax = plt.subplots(figsize=(12, 8))



sns.violinplot(data=df, ax=ax, x='WorkExp')

for q in [0.25, 0.5, 0.75]:
    ax.axvline(df['WorkExp'].quantile(q), **quantile_args)
ax.axvline(df['WorkExp'].mean(), **mean_args)

fig.suptitle('Anos de experiência programando')

plt.show()
```

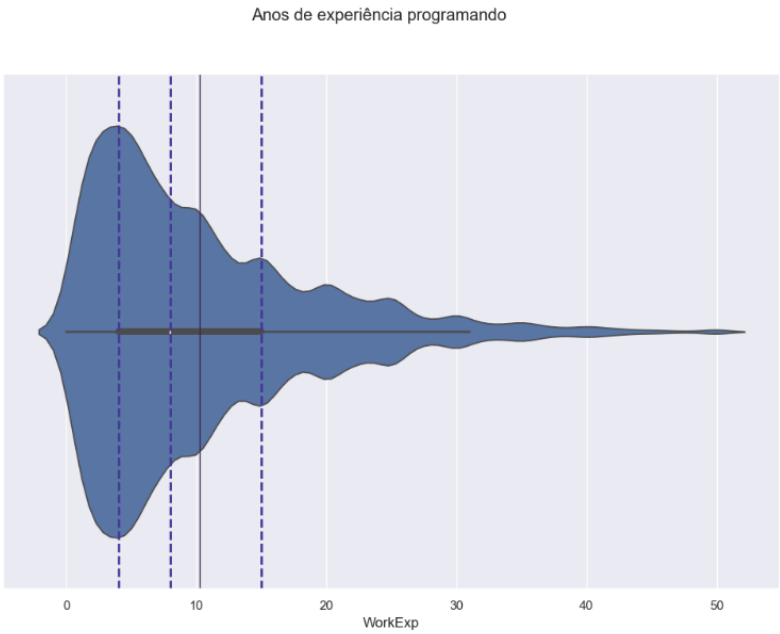


Figure 18: Gráfico violino plotado

Ao contrário do boxplot, que é mais "seco", o gráfico de violino mostra onde há maior concentração de dados com base na largura da curva. Uma parte mais larga indica uma densidade maior naquele intervalo de valores. Isso nos permite entender não apenas os quartis, mas também nuances da distribuição — por exemplo, se os dados estão concentrados em dois picos (bimodalidade).

Podemos tornar essa visualização ainda mais poderosa ao cruzar a variável contínua com uma categórica. A seguir, comparamos a experiência de trabalho (`workexp`) entre diferentes regimes de trabalho remoto:

```
fig, ax = plt.subplots(figsize=(12, 6))
```

```
sns.violinplot(data=df, x='workexp', y='remote_work', ax=ax)
```

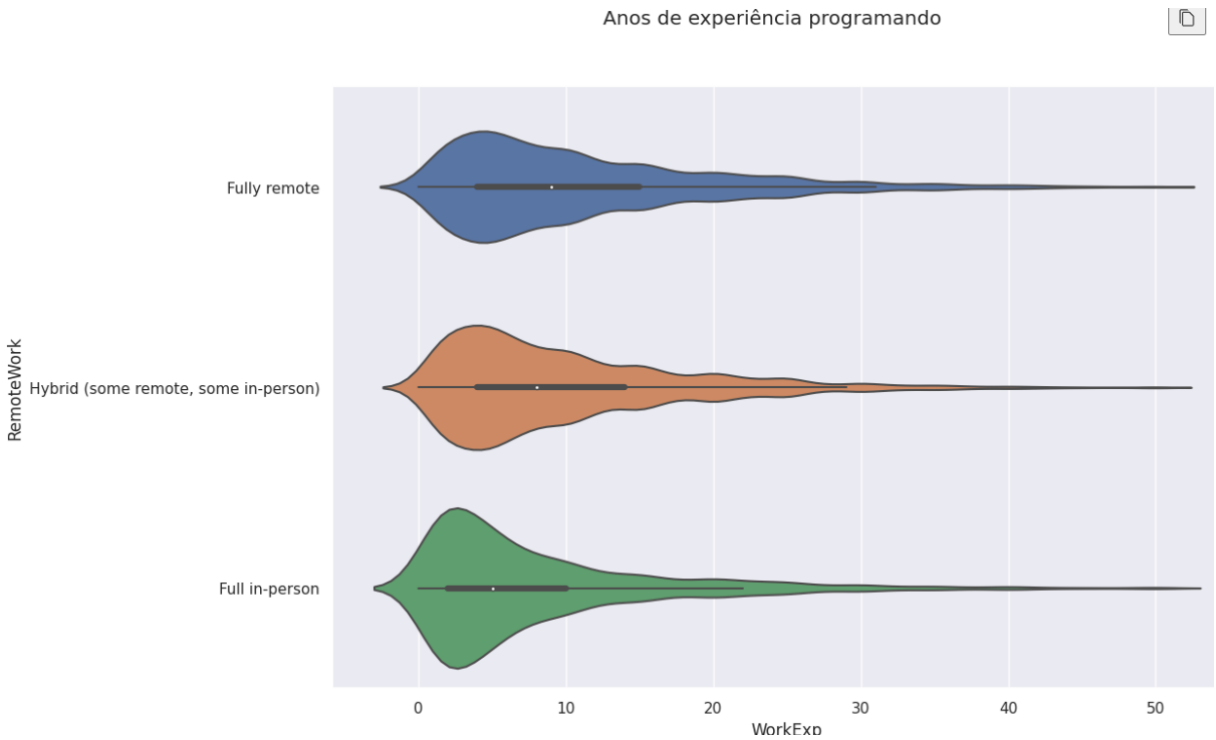


Figure 19: Gráfico violino plotado

Agora o eixo Y representa categorias como "presencial", "híbrido" e "100% remoto". Para cada categoria, é desenhado um "violino" que mostra a distribuição da experiência de trabalho naquela condição. Visualmente, fica claro que os profissionais em regime presencial tendem a ter menos anos de experiência. Isso já levanta hipóteses interessantes sobre a composição das equipes nesses diferentes modelos.

Enriquecendo com Variáveis Segmentadoras

Uma das vantagens dos gráficos de violino é a possibilidade de incluir uma **dimensão adicional de segmentação**, por meio do parâmetro `hue`. Com ele, conseguimos comparar a distribuição de uma variável contínua entre diferentes subgrupos. Por exemplo, podemos investigar se há diferença na experiência de trabalho entre pessoas que já trabalharam com Python e aquelas que nunca trabalharam com a linguagem.

Para isso, criamos uma nova variável booleana (`True` ou `False`) chamada `WorkedWithPython`, que identifica se a pessoa mencionou já ter usado Python profissionalmente. A informação original

está na coluna `LanguageHaveWorkedWith`, que traz todas as linguagens declaradas, separadas por ponto-e-vírgula:

```
df['LanguageHaveWorkedWith'].value_counts()
```

Como cada célula pode conter várias linguagens ao mesmo tempo, usamos `str.contains('Python')` para identificar se o termo "Python" aparece:

```
df['WorkedWithPython'] = df['LanguageHaveWorkedWith'].str.contains('Python')
df['WorkedWithPython'].value_counts(normalize=True, dropna=False).apply(lambda p: f'{100 *
↪  p:.2f}%')
```

Esse comando nos permite confirmar se a nova coluna foi criada corretamente, mostrando a proporção de pessoas com e sem experiência com Python.

Agora, podemos gerar o gráfico de violino com essa segmentação. Usando `hue='WorkedWithPython'` e `split=True`, dividimos cada violino em duas metades, representando os dois grupos. Além disso, para variar um pouco, invertemos os eixos X e Y do gráfico anterior:

```
fig, ax = plt.subplots(figsize=(12, 8))
sns.violinplot(data=df, ax=ax, y='WorkExp', x='RemoteWork', hue='WorkedWithPython', split=True)
fig.suptitle('Anos de experiência programando')
plt.show()
```

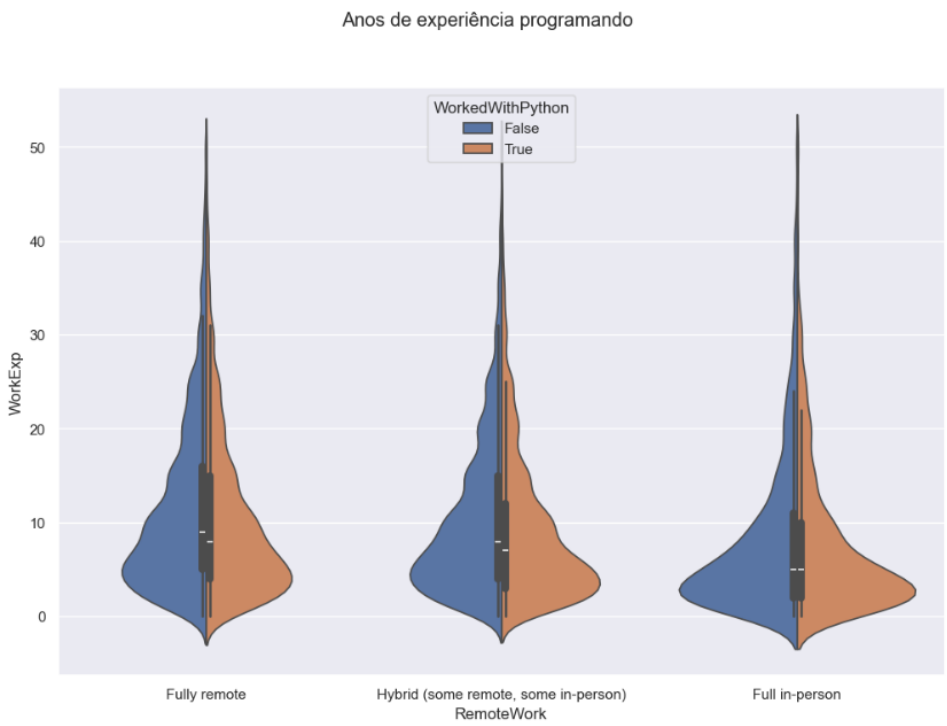


Figure 20: Gráfico violino plotado

Essa visualização revela uma tendência interessante: em todos os regimes de trabalho (remoto, híbrido ou presencial), quem já trabalhou com Python tende a ter um pouco menos de anos de experiência. Isso faz sentido se considerarmos que Python se popularizou principalmente nos últimos anos, impulsionado por áreas como ciência de dados, automação e inteligência artificial — todas relativamente recentes no mercado.

Para fortalecer essa interpretação visual com dados quantitativos, usamos o método `groupby` com `agg`:

```
df.groupby(['RemoteWork', 'WorkedWithPython'])['WorkExp'].agg(['mean', 'median'])
```

Essas estatísticas de média e mediana nos ajudam a validar (ou refutar) hipóteses visuais e garantir que nossa análise não dependa apenas da percepção.

Controlando a Suavização dos Violinos

Uma característica muito útil dos gráficos de violino é a possibilidade de **ajustar o nível de suavização** da curva de densidade, usando o parâmetro bw (bandwidth).

- **Bandwidth alto** → A curva fica mais suave, generalizando a distribuição. Isso é ótimo para visualizar padrões amplos e evitar ruído visual.
- **Bandwidth baixo** → A curva mostra todos os detalhes, incluindo picos e flutuações menores. Pode ser útil para detectar nuances, mas também aumenta o risco de interpretar variações aleatórias como padrões significativos.

Vamos ver isso na prática, comparando duas visualizações com diferentes níveis de suavização:

```
fig, (ax_esq, ax_dir) = plt.subplots(figsize=(22, 8), ncols=2, sharey=True)

sns.violinplot(data=df, ax=ax_esq, y='WorkExp', x='RemoteWork', hue='WorkedWithPython',
↪  split=True, bw=1)  # muito suave

sns.violinplot(data=df, ax=ax_dir, y='WorkExp', x='RemoteWork', hue='WorkedWithPython',
↪  split=True, bw=0.01)  # pouco suave

fig.suptitle('Anos de experiência programando')

plt.show()
```

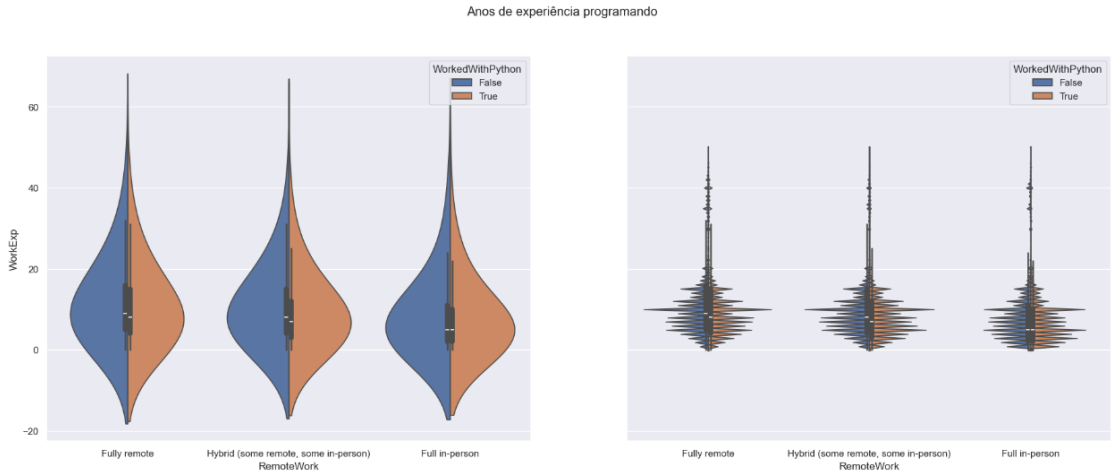


Figure 21: Gráfico violino plotado

No primeiro gráfico (bw=1), as curvas são suaves, ajudando a captar uma visão geral da distribuição. Já no segundo (bw=0.01), os violinos têm contornos irregulares e cheios de picos — revelando mais detalhes, mas também tornando a leitura mais "picotada".

A escolha do bw depende do objetivo da análise: se queremos insights gerais, curvas suaves funcionam melhor; se o foco está nos detalhes, vale a pena testar valores menores. ## Conclusão

O gráfico de violino é uma ferramenta poderosa na visualização de dados, pois une o melhor dos dois mundos: o resumo estatístico do boxplot com a riqueza informativa da curva de densidade. Ele permite observar tanto medidas centrais quanto a forma da distribuição.

Além disso, oferece grande flexibilidade:

- É possível comparar grupos diferentes com hue;
- Podemos ajustar o nível de detalhamento com bw;
- A visualização pode ser customizada para se adaptar a diferentes contextos analíticos.

Por isso, o violinplot é uma excelente escolha em análises exploratórias, especialmente quando buscamos entender nuances que os gráficos tradicionais poderiam esconder.

10. Normalidade e outras métricas de dispersão

Neste capítulo, vamos explorar o conceito de normalidade em conjuntos de dados e outras métricas de dispersão fundamentais: desvio padrão, assimetria e curtose. Também aprenderemos como avaliar a normalidade dos nossos dados e que transformações podem ajudar a aproximá-los de uma distribuição normal.

O que é uma distribuição normal?

A distribuição normal, também chamada de curva de Gauss, é um modelo teórico de distribuição de frequências que apresenta as seguintes características:

- Um pico central que corresponde à média dos dados.
- Simetria: as duas "caudas" descem de forma idêntica para ambos os lados da média.
- Dispersão controlada pelo desvio padrão, que define a largura da curva.

Para visualizar essa curva em Python, podemos usar o módulo `scipy.stats`:

Primeiro, precisaremos instalar a biblioteca `scipy` através do seguinte comando no terminal:

```
pip install scipy
```

Em seguida utilizaremos o seguinte script:

```
import numpy as np
from scipy.stats import norm

xs = np.linspace(-4, 4, 1000)
ys = norm().pdf(xs)

sns.lineplot(x=xs, y=ys)
plt.show()
```

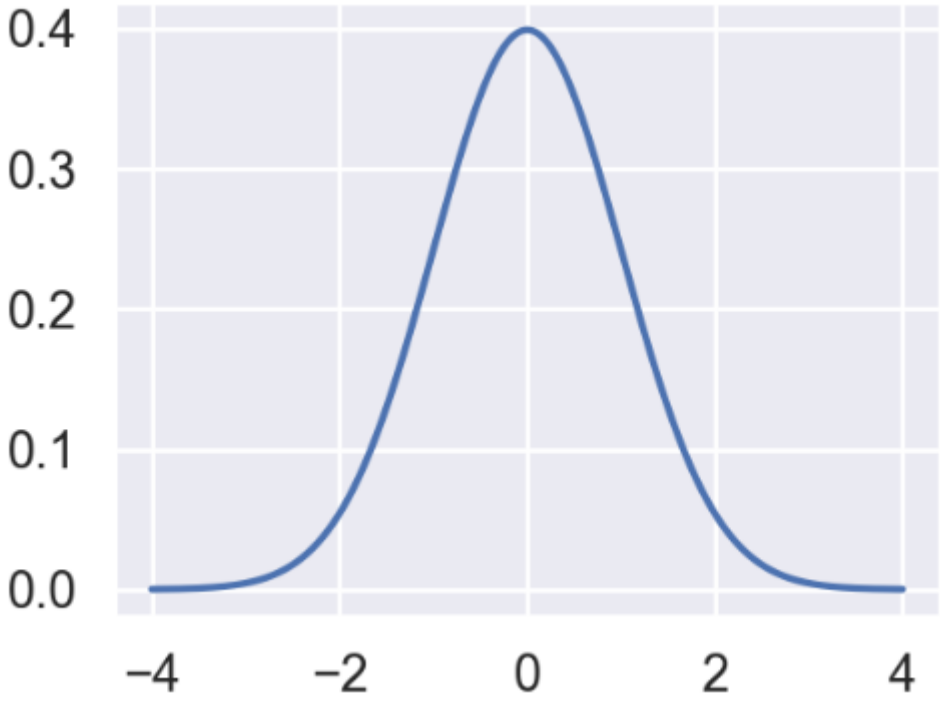


Figure 22: Gráfico distribuição normal

> **Observação:** A função `norm()` retorna uma distribuição normal padrão (média 0, desvio 1). O método `.pdf()` calcula o valor da densidade em cada ponto de `x`.

Parâmetros da curva normal: média e desvio padrão

A curva normal possui dois parâmetros principais:

1. **Média (μ)**: define a posição do pico central da curva.
2. **Desvio padrão (σ)**: controla o grau de espalhamento dos dados ao redor da média.

Podemos ilustrar como esses parâmetros afetam a curva:

```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt

xs = np.linspace(-10, 10, 1000)

# Criar dois subplots lado a lado\ nfig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 4))
```

```
# Variação da média
ax1.plot(xs, norm(loc=0, scale=1).pdf(xs), label='média=0')
ax1.plot(xs, norm(loc=3, scale=1).pdf(xs), label='média=3')
ax1.plot(xs, norm(loc=-6, scale=1).pdf(xs), label='média=-6')
ax1.set_title('Efeito da média (μ)')
ax1.legend()

# Variação do desvio padrão
ax2.plot(xs, norm(loc=0, scale=1).pdf(xs), label='σ=1')
ax2.plot(xs, norm(loc=0, scale=0.5).pdf(xs), label='σ=0.5')
ax2.plot(xs, norm(loc=0, scale=3).pdf(xs), label='σ=3')
ax2.set_title('Efeito do desvio padrão (σ)')
ax2.legend()

plt.tight_layout()
plt.show()
```

Com isso, vemos que aumentar a média desloca a curva horizontalmente e ajustar o desvio padrão altera sua largura e altura.

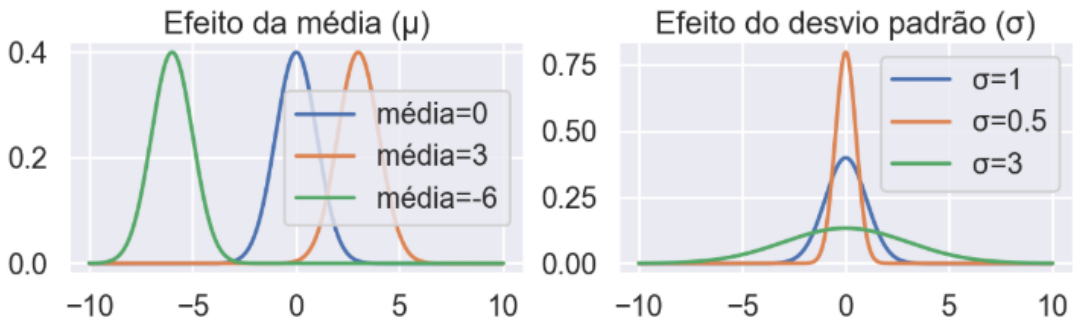


Figure 23: Gráfico distribuição normal

Como avaliar a normalidade dos dados?

Para verificar se os dados seguem uma distribuição normal, podemos:

1. **Inspeção visual**: comparar o histograma dos dados com a curva normal teórica.
2. **Métricas numéricas**: calcular assimetria e curtose para quantificar desvios em relação à normalidade.

Avaliação de normalidade em dados reais

Queremos verificar se a distribuição de `WorkExp` (anos de experiência) em nosso dataset segue uma curva normal.

```
import numpy as np
from scipy.stats import norm

media = df['WorkExp'].mean()
desvio = df['WorkExp'].std()
v_min = df['WorkExp'].min()
v_max = df['WorkExp'].max()

xs = np.linspace(v_min, v_max, 10_000)
ys = norm.pdf(xs, loc=media, scale=desvio)

fig, ax = plt.subplots(figsize=(18, 8))

sns.histplot(data=df, ax=ax, x='WorkExp', binwidth=1.0, kde=True, stat='density')
ax.plot(xs, ys, color='#458134')

fig.suptitle('Anos de experiência programando')
plt.show()
```

Figure 24: Gráfico distribuição normal

Uma das formas de analisar a curva de normalidade, como mencionado na seção anterior, é por meio da inspeção visual. Nesse caso, comparamos a curva dos dados do nosso conjunto (representada pela linha azul) com a curva teórica da distribuição normal padrão, fornecida pela função `norm` da biblioteca `scipy.stats` (representada pela linha verde).

No entanto, ao observar o gráfico, é evidente que as duas curvas apresentam diferenças significativas. Esse desvio indica que os dados do nosso conjunto não seguem uma distribuição normal, o que pode ser resultado de assimetrias, curtose acentuada ou presença de outliers. ## Outras métricas de dispersão: assimetria e curtose

Além do desvio padrão, existem duas métricas muito úteis para entender o formato da distribuição:

- **Assimetria (skewness)**: mede o grau de inclinação da distribuição.
  - Valores > 0: cauda mais longa à direita.
  - Valores < 0: cauda mais longa à esquerda.
  - Na normal ideal, skewness = 0.
- **Curtose (kurtosis)**: mede o nível de concentração dos dados no pico.
  - Valores > 0: pico mais afunilado (leptocúrtica).
  - Valores < 0: curva mais achatada (platicúrtica).
  - Na normal ideal, kurtosis = 0.

Uma distribuição normal possui Assimetria = 0 e Curtose = 0:

```
dist_normal = norm().rvs(10_000)  # Valores sorteados da distribuição normal

print('Assimetria dist. normal:', pd.Series(dist_normal).skew())
print('Curtose dist. normal:', pd.Series(dist_normal).kurtosis())
```

Rodando o script acima , você perceberá que receberemos valores muito próximos de 0

Vamos medir estes valores para nossos dados, e compará-los com a distribuição normal:

```
dist_normal = norm().rvs(10_000)  # Valores sorteados da distribuição normal

print('Assimetria dados:', df['WorkExp'].skew())
print('Assimetria dist. normal:', pd.Series(dist_normal).skew())
print('-----')
print('Curtose dados:', df['WorkExp'].kurtosis())
print('Curtose dist. normal:', pd.Series(dist_normal).kurtosis())
```

```
Assimetria dados: 1.4409481289144255
Assimetria dist. normal: -0.032560591380274014
-----
Curtose dados: 2.1453144700502444
Curtose dist. normal: 0.01931302743960872
```

Figure 25: Log com o resultado comparativo

Transformações para aproximar da normalidade

Quando os dados não são normais, é comum aplicar transformações para atenuar assimetrias e picos:

- **Transformação logarítmica**: recomendada para distribuições com assimetria positiva.

```
import numpy as np
from scipy.stats import norm
# Filtrar zeros para evitar indefinição em log(0)
work_exp = df['WorkExp'].loc[df['WorkExp'] > 0]

df['LogWorkExp'] = np.log(work_exp)

media_log = df['LogWorkExp'].mean()
desvio_log = df['LogWorkExp'].std()
v_min_log = df['LogWorkExp'].min()
v_max_log = df['LogWorkExp'].max()

xs = np.linspace(v_min_log, v_max_log, 10_000)
ys = norm.pdf(xs, loc=media_log, scale=desvio_log)

fig, ax = plt.subplots(figsize=(18, 8))

sns.histplot(data=df, ax=ax, x='LogWorkExp', kde=True, stat='density')
ax.plot(xs, ys, color='#458134')
fig.suptitle('Anos de experiência programando')



plt.show()
```

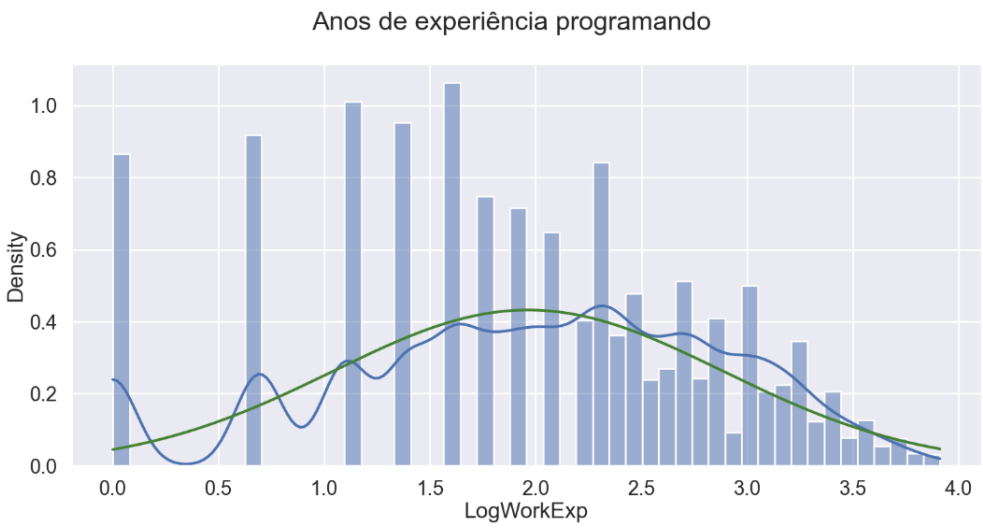


Figure 26: Gráfico distribuição normal

A distribuição dos dados ainda não se comporta exatamente como a curva normal, mas está perceptivelmente, muito mais próxima ## Conclusão

Neste capítulo. vimos como identificar a normalidade dos dados visualmente e numericamente, além de outras métricas de dispersão que complementam o desvio padrão. Aprendemos também a aplicar uma transformação logarítmica para tornar distribuições enviesadas mais próximas de uma curva normal. Essas técnicas são fundamentais para garantir que testes e modelos estatísticos que assumem normalidade tenham resultados mais robustos.

11. Conceito de Correlação

Dizemos que duas variáveis A e B são **correlacionadas** quando, em média, o aumento de A está associado ao aumento ou diminuição de B. Existem três tipos principais de correlação:

- **Correlação positiva**: A e B tendem a crescer simultaneamente.
- **Correlação negativa**: A cresce enquanto B diminui.
- **Sem correlação**: Não há relação aparente entre os valores de A e B.

> **Exemplo**: Altura × Massa corporal. Pessoas mais altas tendem, em média, a ter massa corporal maior, representando uma correlação positiva. Entretanto, correlação não implica causa e efeito.

Visualizando Correlações com Gráficos de Dispersão

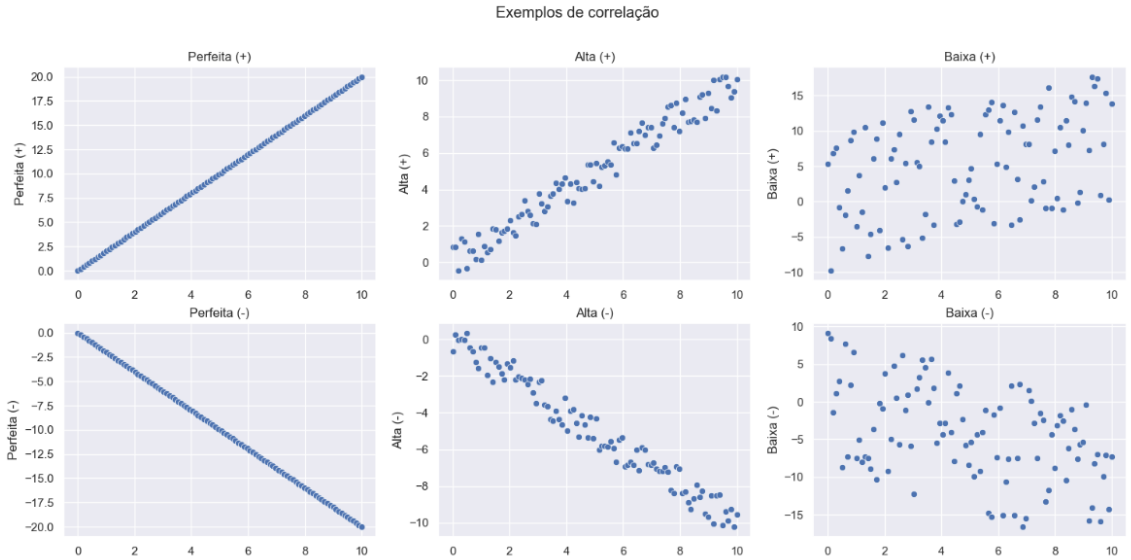
Gráficos de dispersão são uma forma prática de visualizar a relação entre duas variáveis. Cada ponto no gráfico representa um par (x, y).

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
sns.set_context('notebook')

# Gerando dados de exemplo
x = np.linspace(0, 10, 100)
y_random = np.random.uniform(-1, 1, 100)      # Sem correlação
y_corr = x * 2 + np.random.uniform(-1, 1, 100)  # Correlação alta

# Plotando side-by-side
fig, (ax_esq, ax_dir) = plt.subplots(figsize=(18, 8), ncols=2)

sns.scatterplot(ax=ax_esq, x=x, y=y_corr)
ax_esq.set_title('Correlação alta')

sns.scatterplot(ax=ax_dir, x=x, y=y_random)
ax_dir.set_title('Sem correlação')

fig.suptitle('Exemplos de correlação')
plt.show()
```

No gráfico à esquerda, vemos pontos alinhados em torno de uma reta ascendente (correlação alta). À direita, os pontos estão espalhados sem padrão definido (sem correlação). ## Variações de Intensidade de Correlação

Podemos gerar diferentes intensidades de correlação (perfeita, alta, baixa) e também exemplos de correlação negativa:

```
import itertools

x = np.linspace(0, 10, 100)
df_corr = pd.DataFrame({
    'Perfeita (+)': x * 2,
    'Alta (+)': x + np.random.uniform(-1, 1, 100),
    'Baixa (+)': x + np.random.uniform(-10, 10, 100),
    'Perfeita (-)': -x * 2,
    'Alta (-)': -x + np.random.uniform(-1, 1, 100),
    'Baixa (-)': -x + np.random.uniform(-10, 10, 100),
})

fig, axes = plt.subplots(figsize=(18, 8), ncols=3, nrows=2)
for column, ax in zip(df_corr, itertools.chain.from_iterable(axes)):
    sns.scatterplot(ax=ax, x=x, y=df_corr[column])
    ax.set_title(column)
fig.suptitle('Exemplos de correlação')
plt.show()
```



Importante: correlação é meramente um vínculo matemático. Não implica necessariamente causa e efeito! Por exemplo: Ter mais de um banheiro em casa está correlacionado a uma renda mais alta. Isso não significa que meu salário vai aumentar se eu construir um novo banheiro! - **Correlação perfeita**: todos os pontos alinhados em uma reta. - **Correlação alta**: pontos em torno de uma reta, com dispersão moderada. - **Correlação baixa**: maior dispersão, mas ainda com leve tendência. - **Correlação negativa**: mesmos padrões, invertendo a inclinação.

Medindo Correlação com Pearson R

Para quantificar a força de correlação usamos o **coeficiente de correlação de Pearson (r)**, que varia de -1 a 1. Podemos calculá-lo com `pearsonr` do `scipy.stats`:

```
from scipy.stats import pearsonr

for column in df_corr:
    r, _ = pearsonr(x, df_corr[column])
    print(f'Correlação entre x e {column}: {r:.3f}')
```

- **r = 1.0**: correlação positiva perfeita
- **r ≈ 0**: sem correlação
- **r = -1.0**: correlação negativa perfeita

Valores intermediários indicam correlação moderada ou fraca.

```
A correlação entre X e coluna Correlação perfeita (+) é 1.000
A correlação entre X e coluna Correlação alta (+) é 0.982
A correlação entre X e coluna Correlação baixa (+) é 0.493
A correlação entre X e coluna Correlação perfeita (-) é -1.000
A correlação entre X e coluna Correlação alta (-) é -0.980
A correlação entre X e coluna Correlação baixa (-) é -0.431
```

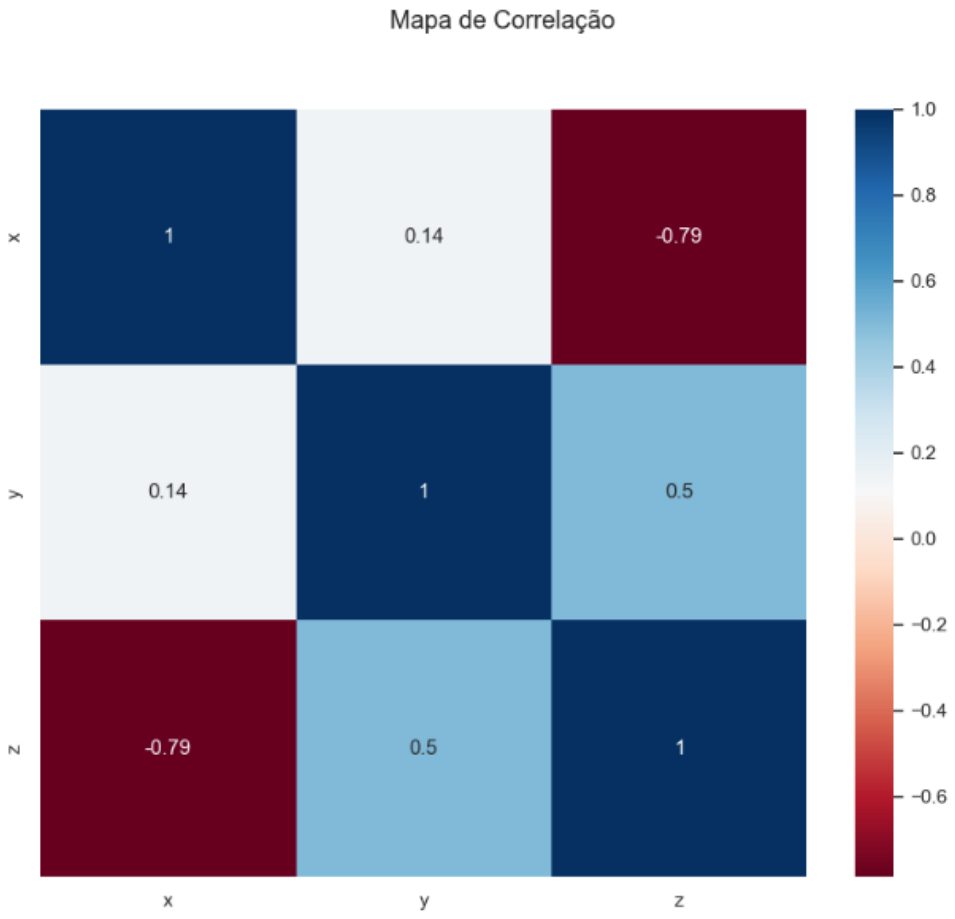
Correlação em DataFrames e Heatmap

Para calcular correlações entre todas as colunas numéricas de um DataFrame de uma só vez, usamos:

```
sample = pd.DataFrame({
    'x': [10, 5, 2],
    'y': [9, 8, 9],
    'z': [4, 4, 7],
})

corr_matrix = sample.corr()
print(corr_matrix)
```

Para visualizar de forma gráfica, utilizamos um **heatmap**:

```
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(data=sample.corr(), square=True, cmap='RdBu', annot=True)
fig.suptitle('Mapa de Correlação')
plt.show()
```



O heatmap destaca, com cores e anotações, as correlações positivas (tons de azul) e negativas (tons de vermelho), facilitando a identificação de padrões em DataFrames com muitas colunas.

Resumo

- Correlação indica associação estatística entre variáveis, sem implicar causalidade.
- Gráficos de dispersão permitem visualizar diferentes intensidades e direções de correlação.
- O coeficiente de Pearson quantifica a força de correlação em um valor entre -1 e 1.
- Para múltiplas variáveis, `df.corr()` e `sns.heatmap` geram uma visão geral eficiente em forma de matriz colorida.

Conclusão

Neste capítulo, aprendemos a gerar e interpretar gráficos de dispersão, calcular correlações e a usar heatmaps para explorar DataFrames extensos.

12. Correlação nos nossos dados

Neste capítulo, vamos investigar se há alguma relação entre os anos de experiência profissional e os salários anuais das pessoas desenvolvedoras que responderam à pesquisa. A ideia é aplicar o conceito de **correlação** que vimos anteriormente e usá-lo para explorar padrões reais nos dados — e, claro, também identificar os desafios práticos, como a presença de **outliers** e a necessidade de segmentar melhor os dados.

Retomando: o que é Correlação?

Correlação é uma medida estatística que indica a força e a direção de uma relação linear entre duas variáveis. Varia de –1 (correlação negativa perfeita) a +1 (correlação positiva perfeita). Valores próximos de 0 indicam pouca ou nenhuma relação linear.

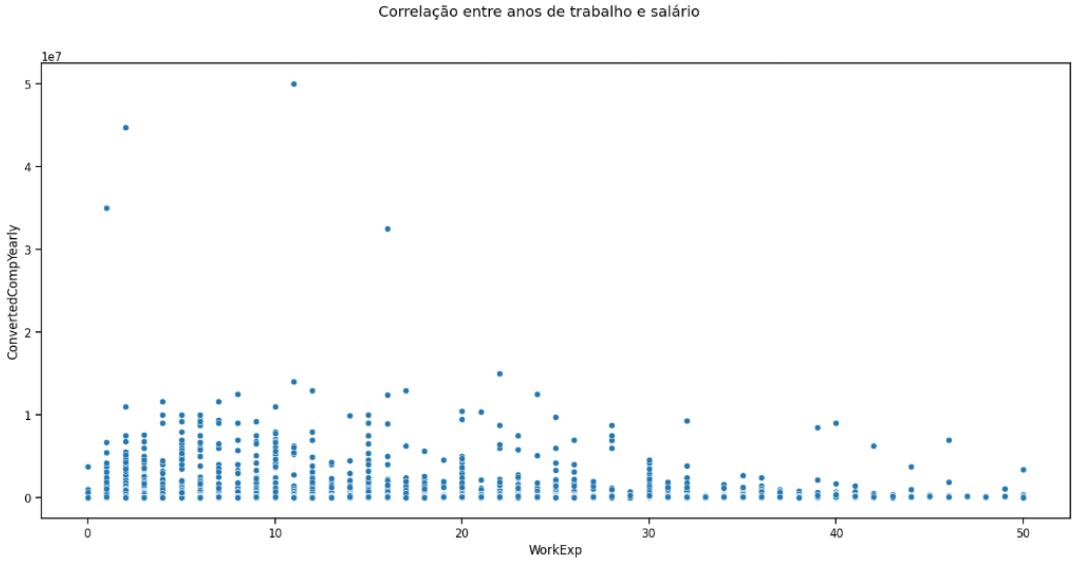
Por que Filtrar Outliers?

Outliers são pontos extremos que podem distorcer a escala do gráfico e esconder padrões nos dados principais. Ao removê-los (por exemplo, salários acima de 1 000 000 USD/ano), focamos no comportamento da maioria dos respondentes.

Explorando os dados: salário e experiência

A pergunta que nos guia aqui é simples: **será que pessoas desenvolvedoras ganham mais conforme acumulam mais anos de experiência?** Vamos usar um gráfico de dispersão para visualizar essa possível relação.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carregando os dados
df = pd.read_csv('materiais de aula/survey_results_public.csv', index_col='ResponseId')

# Configurações de visualização
sns.set_context('notebook')

# Scatterplot inicial
fig, ax = plt.subplots(figsize=(18, 8))
sns.scatterplot(data=df, x='WorkExp', y='ConvertedCompYearly', ax=ax)
fig.suptitle('Correlação entre anos de trabalho e salário')
plt.show()
```

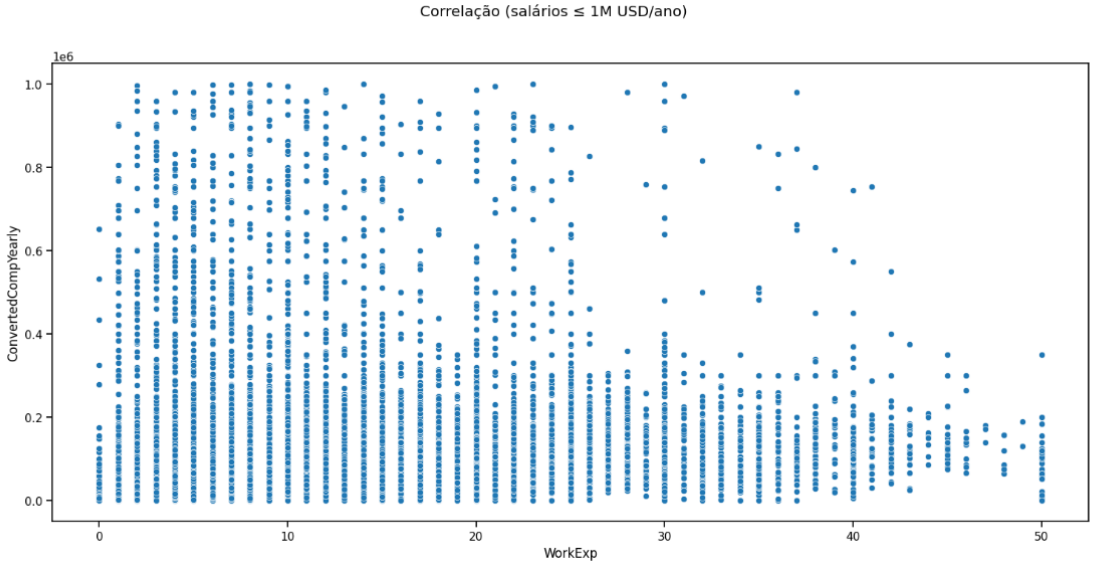


Observação: O gráfico revela uma grande concentração de salários abaixo de 300 mil dólares por ano, com alguns poucos valores extremos (outliers) distorcendo a visualização.

Lidando com Outliers

Quando olhamos para os dados brutos, vemos que há valores altíssimos, chegando a milhões de dólares por ano. Esses valores são raros e, muitas vezes, comprometem a análise visual e estatística. Vamos aplicar um filtro para remover salários acima de 1 milhão de dólares:

```
# Filtrando salários acima de 1 milhão USD
df_filter = df.loc[df['ConvertedCompYearly'] <= 1_000_000].copy()
# Novo gráfico
fig, ax = plt.subplots(figsize=(18, 8))
sns.scatterplot(data=df_filter, x='WorkExp', y='ConvertedCompYearly', ax=ax)
fig.suptitle('Correlação (salários ≤ 1M USD/ano)')
plt.show()
```



Agora conseguimos ver com mais clareza a distribuição principal dos dados. Ainda assim, parece haver dois grupos distintos: um de salários "altos" e outro de salários "médios".

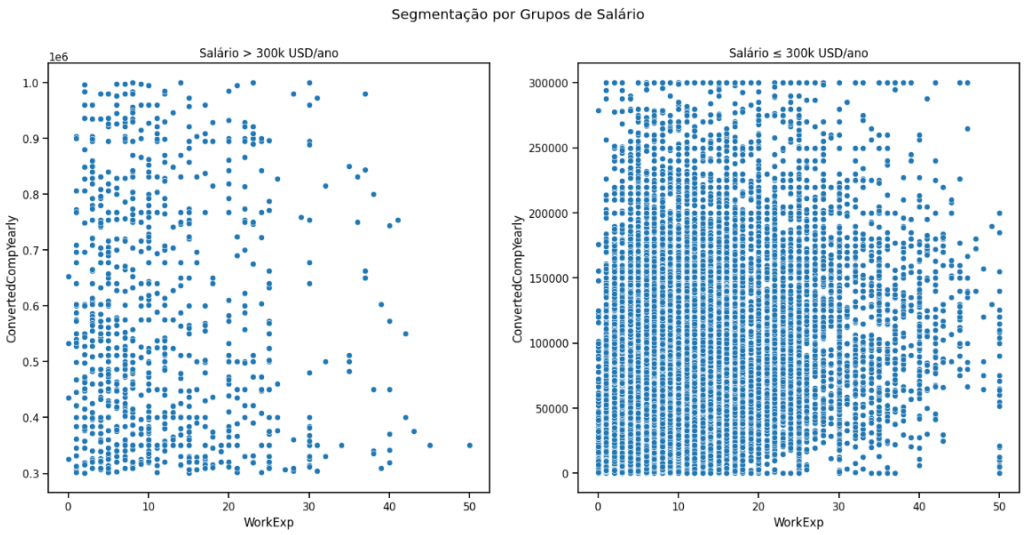
Segmentando os dados por faixa salarial

Vamos separar o conjunto filtrado em dois grupos: um com salários acima de 300 mil dólares e outro com salários abaixo ou iguais a esse valor.

```
cutoff = 300_000
upper_df = df_filter.loc[df_filter['ConvertedCompYearly'] > cutoff]
lower_df = df_filter.loc[df_filter['ConvertedCompYearly'] <= cutoff]
```

Agora, visualizamos esses dois grupos lado a lado:

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18, 8))
sns.scatterplot(data=upper_df, x='WorkExp', y='ConvertedCompYearly', ax=ax1)
ax1.set_title('Salário > 300k USD/ano')

sns.scatterplot(data=lower_df, x='WorkExp', y='ConvertedCompYearly', ax=ax2)
ax2.set_title('Salário ≤ 300k USD/ano')

fig.suptitle('Segmentação por Grupos de Salário')
plt.show()
```



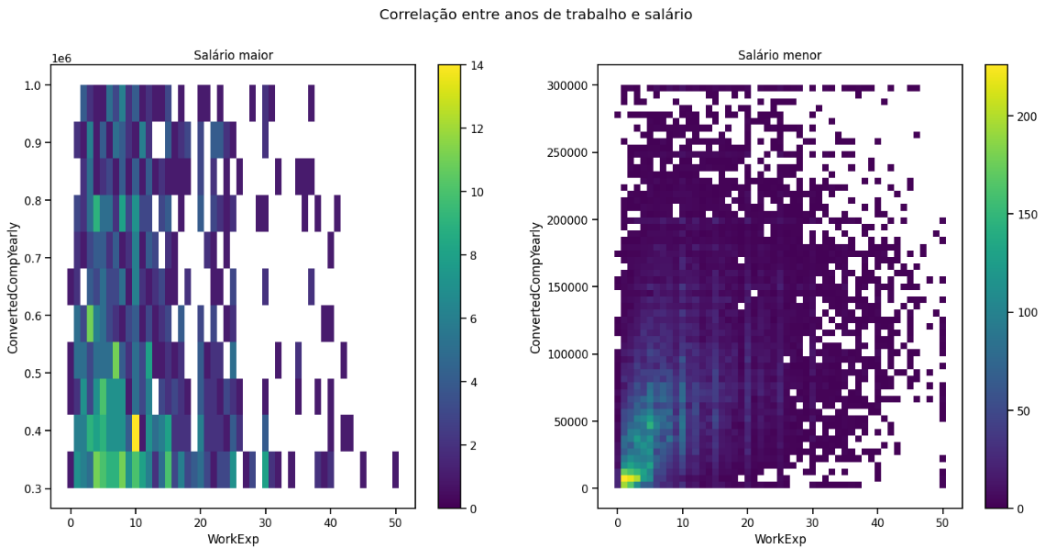
Calculando a correlação nos dois grupos

```
print('Correlação no grupo de salário menor:',
      lower_df[['WorkExp','ConvertedCompYearly']].corr().iloc[0,1])

print('Correlação no grupo de salário maior:',
      upper_df[['WorkExp','ConvertedCompYearly']].corr().iloc[0,1])
```

Resultado: No grupo de salário menor, parece haver uma correlação positiva, ainda que baixa. Isso significa que é mais difícil encontrar pessoas com pouca experiência ganhando muito, ou com muita experiência ganhando pouco. Já no grupo de salário maior, a correlação é praticamente zero.

Visualização com histogramas 2D

Para entender melhor a **densidade dos pontos**, podemos usar histogramas bidimensionais:



Quanto mais escura a diagonal no gráfico, mais forte a tendência de correlação.

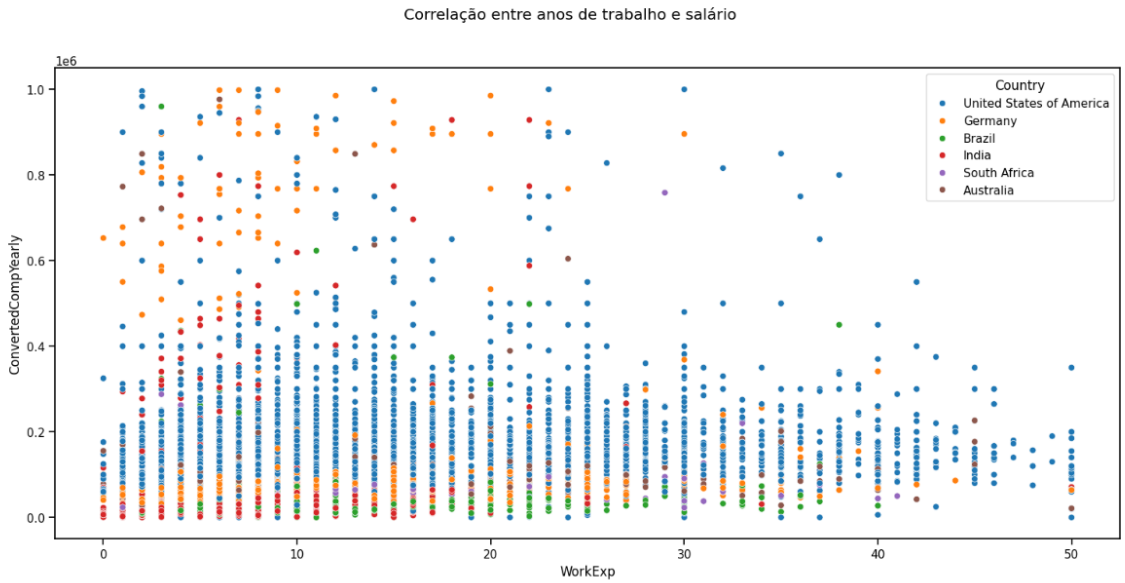
Correlação por país

A correlação entre salário e experiência pode variar de acordo com o país. Vamos investigar isso agora.

```
# Selecionando países
paises = ['United States of America', 'Germany', 'Brazil', 'South Africa', 'India', 'Australia']
df_paises = df_filter.loc[df_filter['Country'].isin(paises)].copy()
print(df_paises['Country'].value_counts())
```

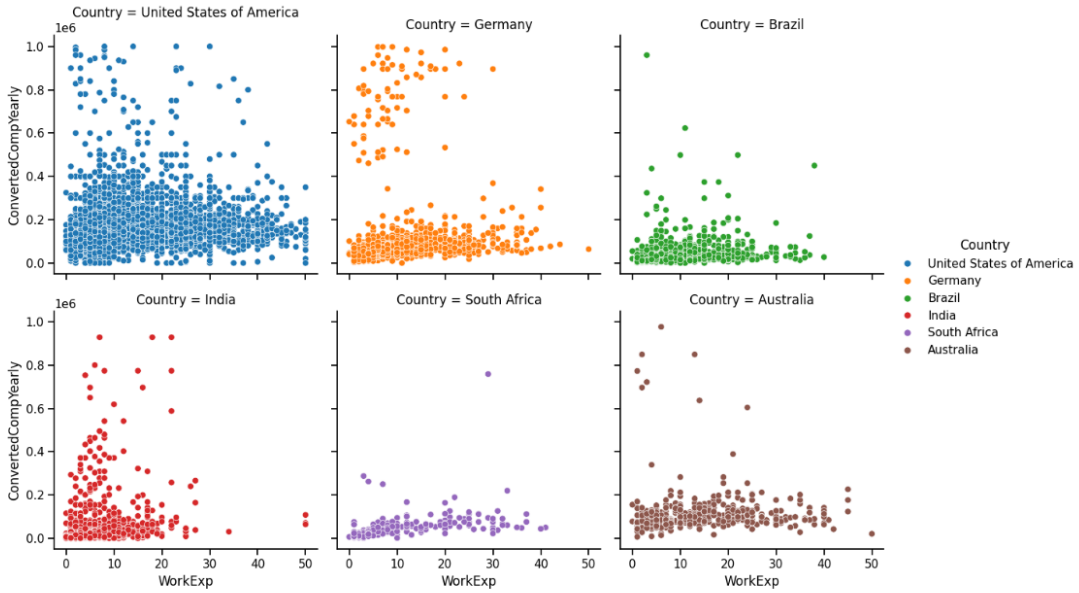
Grande parte das respostas veio dos EUA, mas ainda assim temos um bom número de respostas para cada país. Eis a correlação usando o parâmetro hue para categorizar cada ponto:

```
fig, ax = plt.subplots(figsize=(18, 8))

sns.scatterplot(data=df_paises, ax=ax, x='WorkExp', y='ConvertedCompYearly', hue='Country')

fig.suptitle('Correlação entre anos de trabalho e salário')

plt.show()
```



Visualizamos os dados segmentados por país com a ajuda do `relplot`:

```
g = sns.relplot(data=df_paises, x='WorkExp', y='ConvertedCompYearly',
                col='Country', hue='Country', col_wrap=3, height=4)
plt.show()
```



Agora, calculamos a correlação para cada país: 1) Selecionar apenas as colunas de interesse 2) Agrupar os dados por país 3) Calcular a correlação

```
df_paises[['WorkExp', 'ConvertedCompYearly', 'Country']].groupby('Country').corr()
```

		WorkExp	ConvertedCompYearly
Country			
Australia	WorkExp	1.000000	0.054412
	ConvertedCompYearly	0.054412	1.000000
Brazil	WorkExp	1.000000	0.186447
	ConvertedCompYearly	0.186447	1.000000
Germany	WorkExp	1.000000	0.090473
	ConvertedCompYearly	0.090473	1.000000
India	WorkExp	1.000000	0.224852
	ConvertedCompYearly	0.224852	1.000000
South Africa	WorkExp	1.000000	0.345736
	ConvertedCompYearly	0.345736	1.000000
United States of America	WorkExp	1.000000	0.131599
	ConvertedCompYearly	0.131599	1.000000

Para cada país, foi calculado a matriz de correlação 2x2. Mas na realidade, só temos interesse em 1 valor dessa matriz. Podemos usar `iloc` para selecionar apenas algumas linhas e colunas, simplificando o output:

```
df_paises[['WorkExp', 'ConvertedCompYearly', 'Country']].groupby('Country').corr().iloc[1::2,
↪  0].sort_values(ascending=False)
```

```
Country
South Africa              ConvertedCompYearly    0.345736
India                     ConvertedCompYearly    0.224852
Brazil                    ConvertedCompYearly    0.186447
United States of America  ConvertedCompYearly    0.131599
Germany                   ConvertedCompYearly    0.090473
Australia                 ConvertedCompYearly    0.054412
Name: WorkExp, dtype: float64
```

Conclusão

Esses resultados reforçam a importância de filtrar e segmentar os dados para evitar conclusões enganosas e mostram como a estatística pode ser uma ferramenta poderosa para entender dinâmicas do mercado de trabalho de forma mais realista e aprofundada.

13. Correlação e distribuição de dados

Existe outra função "especial" do `seaborn`, capaz de plotar correlações e distribuições dos nossos dados numéricos de uma única vez.

Preparando os Dados

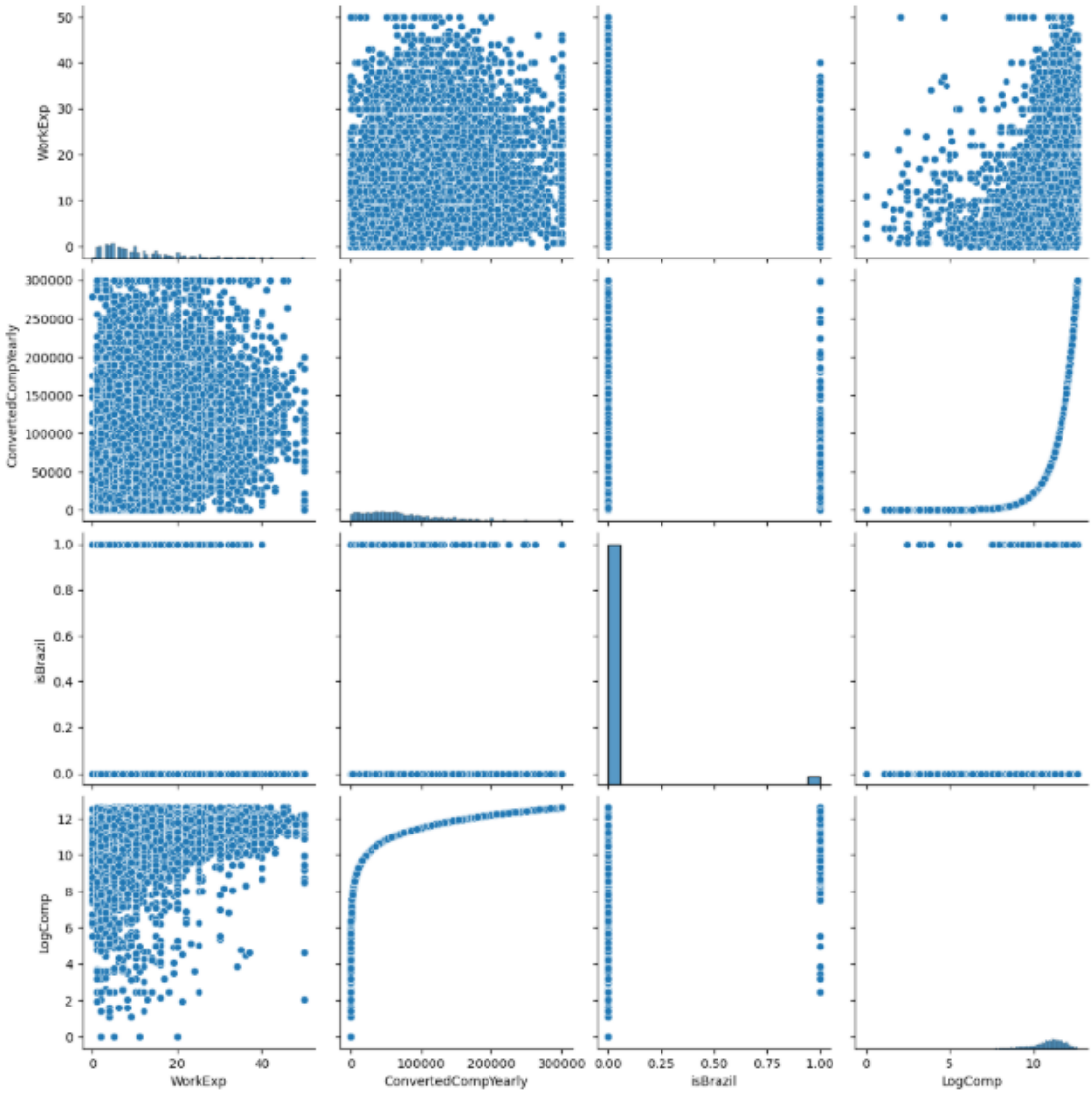
Antes de qualquer visualização, vamos preparar o conjunto de dados para garantir que estamos trabalhando com colunas numéricas limpas e relevantes:

```
import numpy as np

# Criando uma coluna binária indicando se o país é Brasil
df['isBrazil'] = (df['Country'] == 'Brazil').astype(int)

# Criando uma coluna com o logaritmo da compensação anual
df['LogComp'] = np.log(df['ConvertedCompYearly'])

# Filtrando os dados: apenas compensações até 300 mil dólares
df_filtered = df.loc[df['ConvertedCompYearly'] <= 300_000][[
    'WorkExp', 'ConvertedCompYearly', 'isBrazil', 'LogComp'
]]
```

Esse filtro ajuda a remover **outliers extremos** que poderiam distorcer as análises de correlação. ## Pairplot: Correlação Multivariada

O `pairplot` é uma função poderosa para observar a relação entre **todas as combinações possíveis de colunas numéricas** de um DataFrame.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(data=df_filtered, height=3)
plt.show()
```



Como interpretar

O `sns.pairplot` é uma ferramenta bastante útil para a análise exploratória de dados, pois gera uma espécie de "matriz" de gráficos envolvendo todas as colunas numéricas do conjunto de dados. Nessa matriz, cada combinação de duas variáveis é representada por um gráfico de dispersão, permitindo observar possíveis correlações entre elas. Já os gráficos localizados na diagonal principal da matriz mostram histogramas individuais de cada variável, o que facilita a visualização da distribuição dos dados.

No entanto, é importante ficar atento ao número de colunas numéricas presentes no dataset. Quando são muitas, a geração dos gráficos pode se tornar muito demorada e, além disso, o excesso de infor-

mações faz com que os gráficos fiquem extremamente pequenos, dificultando a interpretação visual. Por isso, o ideal é trabalhar com, no máximo, cinco colunas para garantir uma análise mais clara e eficiente.

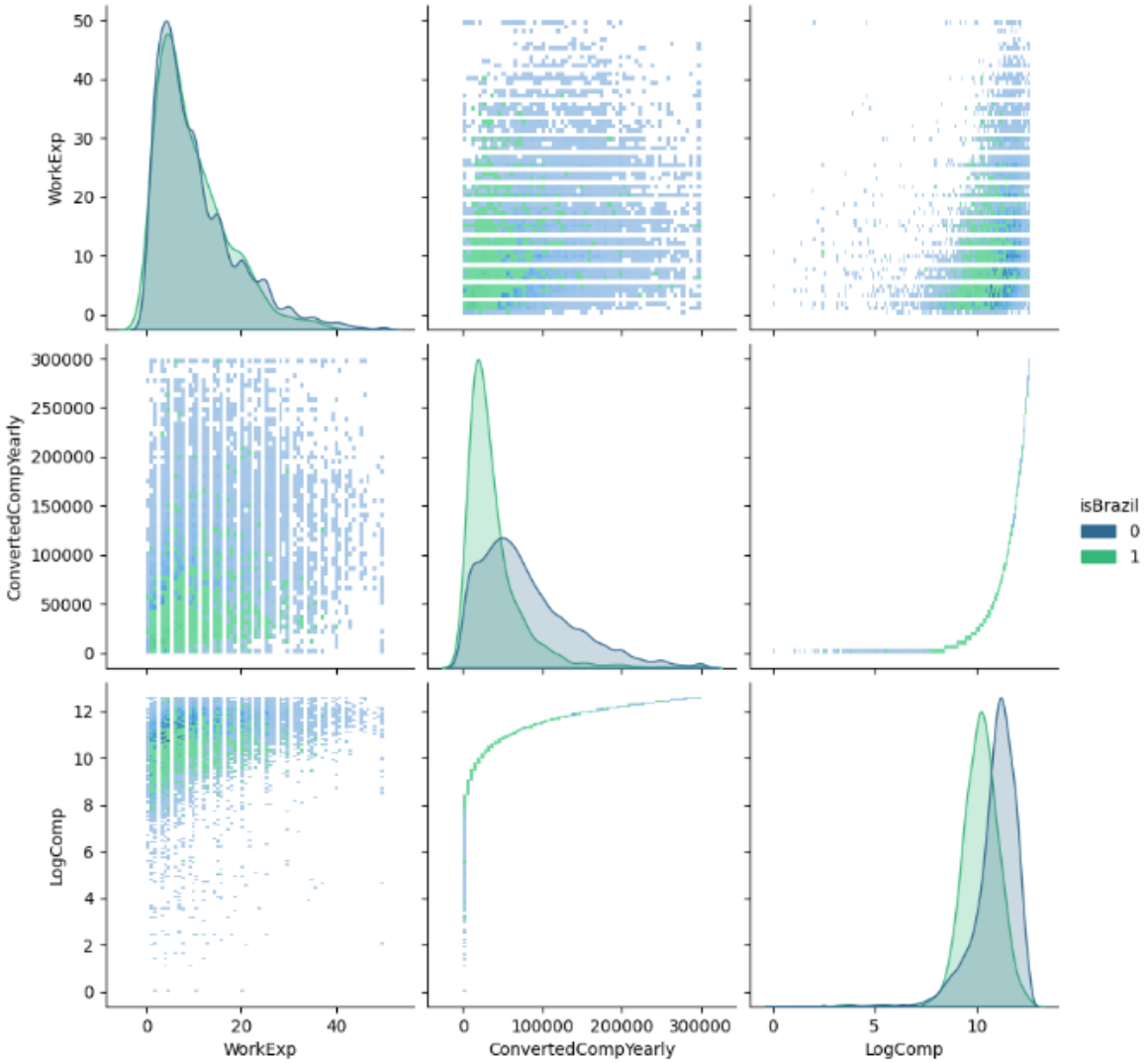
Customizando o Pairplot

Podemos enriquecer o gráfico com várias personalizações, como:

- Dividir os dados por grupos com o parâmetro hue
- Substituir os histogramas da diagonal por curvas KDE
- Usar histogramas 2D no lugar de dispersões

Exemplo com personalização:

```
sns.pairplot(
    data=df_filtered,
    height=3,
    hue='isBrazil',                         # Cores diferentes para Brasil e outros países
    palette='viridis',
    kind='hist',                            # Histogramas 2D fora da diagonal
    diag_kind='kde',                        # Curvas KDE na diagonal
    diag_kws={'common_norm': False}         # Remove normalização comum entre os grupos
)
plt.show()
```



O gráfico agora mostra não apenas a dispersão e distribuição, mas também **as diferenças entre os grupos** (Brasil vs. outros países).

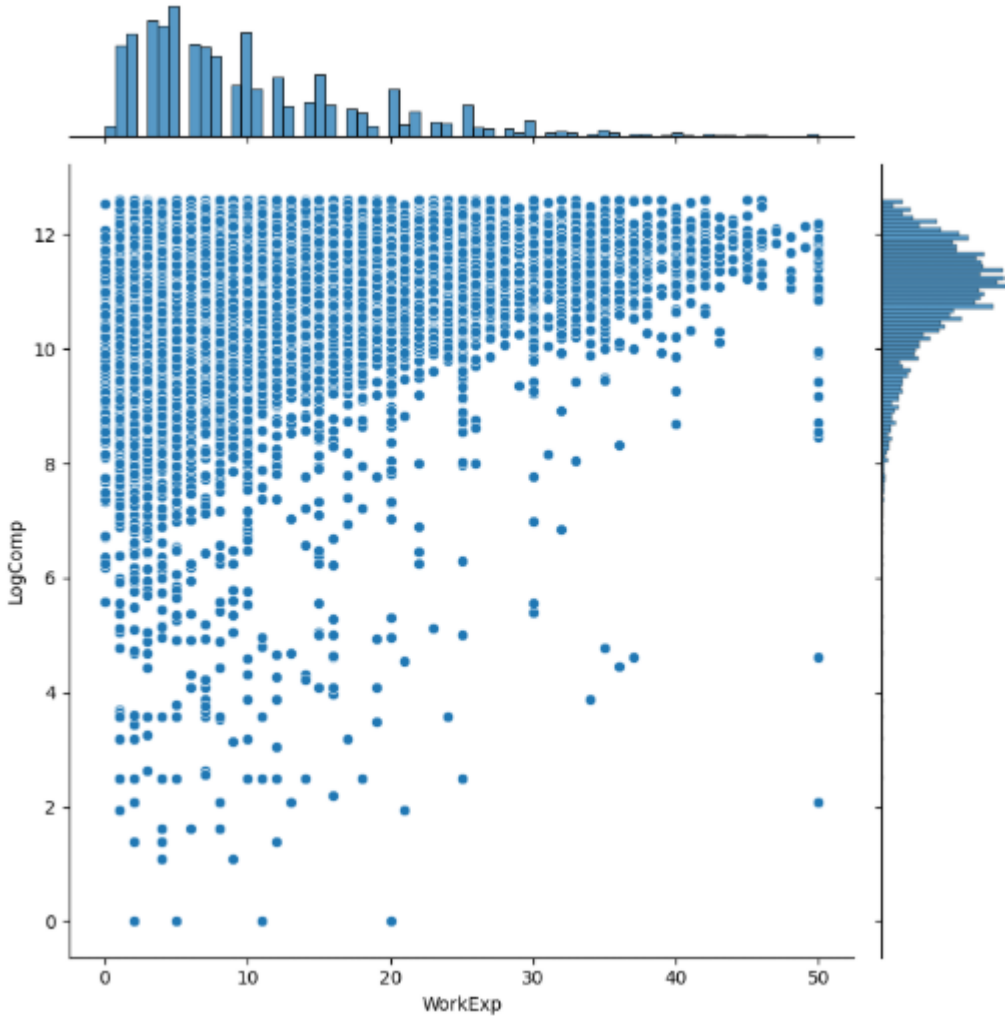
Jointplot: Foco em Duas Variáveis

Se o objetivo for explorar apenas **duas variáveis**, o `jointplot` é mais apropriado. Ele mostra:

- Um gráfico de dispersão no centro
- Histogramas marginais nas bordas

Exemplo:

```
sns.jointplot(data=df_filtered, x='WorkExp', y='LogComp', height=8)
plt.show()
```



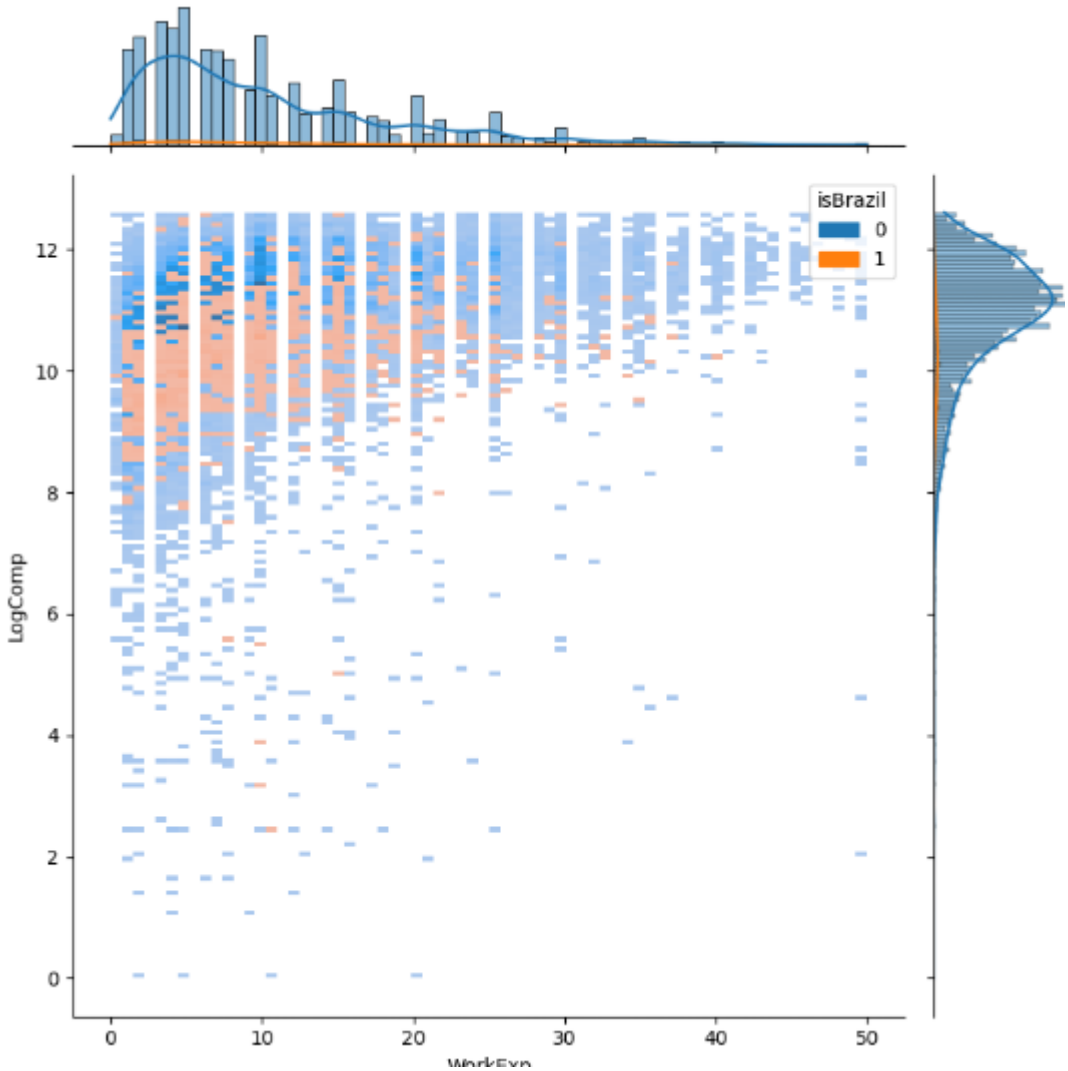
Isso permite observar tanto a correlação entre as duas variáveis quanto suas distribuições individuais. Podemos perceber que este gráfico nada mais é do que o mesmo gráfico do canto inferior esquerdo, com as distribuições de cada variável nas margens.

Da mesma forma como o gráfico anterior, podemos customizar este gráfico também:

Versão com customização:

```
sns.jointplot(
    data=df_filtered,
    x='WorkExp',
```

```
    y='LogComp',
    hue='isBrazil',
    height=8,
    kind='hist',
    marginal_kws={'kde': True}
)
plt.show()
```

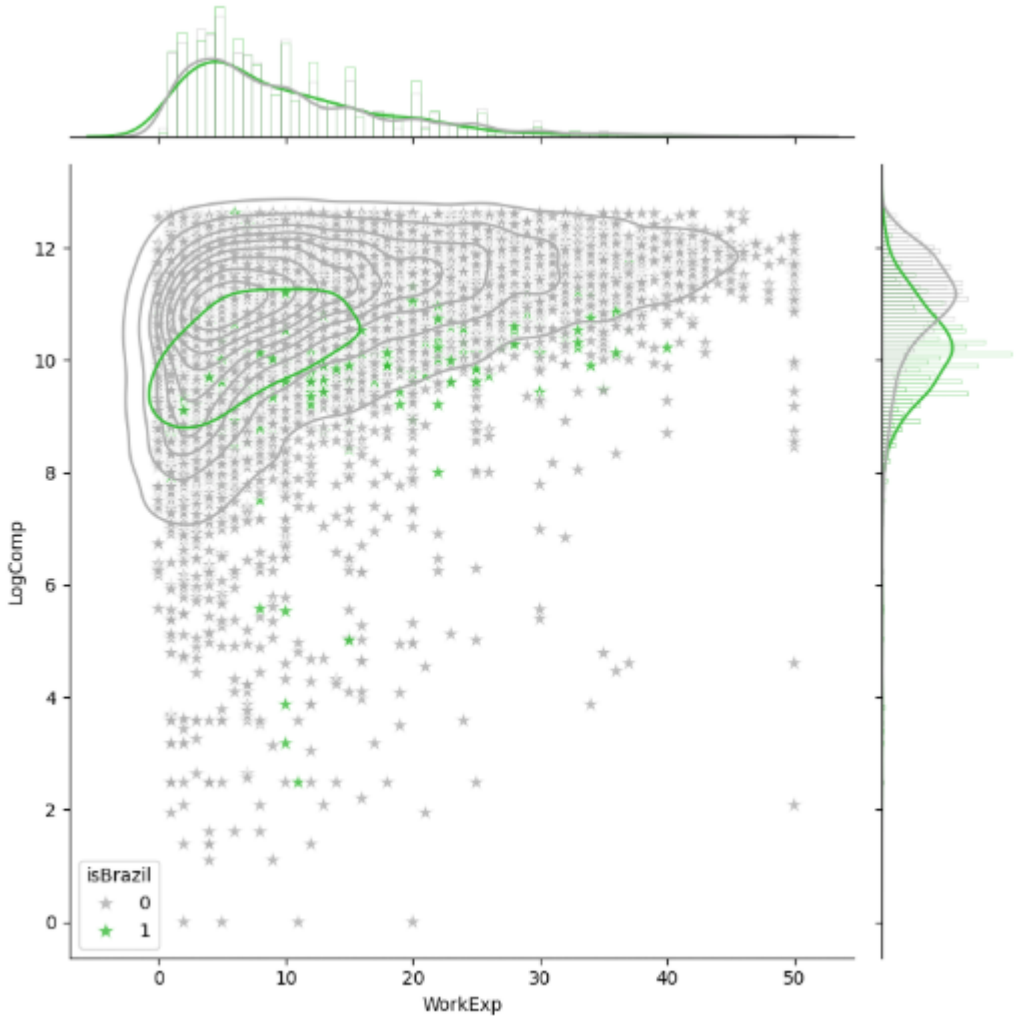


Avançado: Adicionando Camadas ao Jointplot

Com `jointplot`, você pode adicionar camadas extras de visualização, aproveitando a flexibilidade do objeto retornado (`FacetGrid`):

```
g = sns.jointplot(
    data=df_filtered,
```

```
    x='WorkExp',
    y='LogComp',
    hue='isBrazil',
    height=8,
    palette=['#acacac', '#34ba34'],
    hue_order=[0, 1],
    kind='kde',
    marginal_kws={'common_norm': False},
)

# Adicionando um scatterplot central com estrelas
g.plot_joint(sns.scatterplot, alpha=0.8, marker='*', s=100)

# Adicionando histogramas nas margens
g.plot_marginals(sns.histplot, fill=False, stat='density', common_norm=False, alpha=0.5)

plt.show()
```



Essa técnica é especialmente útil quando você deseja enfatizar diferentes aspectos dos dados numa mesma visualização. ## Conclusão

Tanto o `pairplot` quanto o `jointplot` são ferramentas fundamentais para a **exploração visual de dados numéricos**. Eles facilitam a identificação de correlações, outliers, padrões e agrupamentos, mesmo em estágios iniciais da análise.

Use essas visualizações com inteligência, sempre considerando:

- A quantidade de variáveis que está sendo exibida
- A legibilidade dos dados
- A clareza na separação por grupos (quando aplicável).

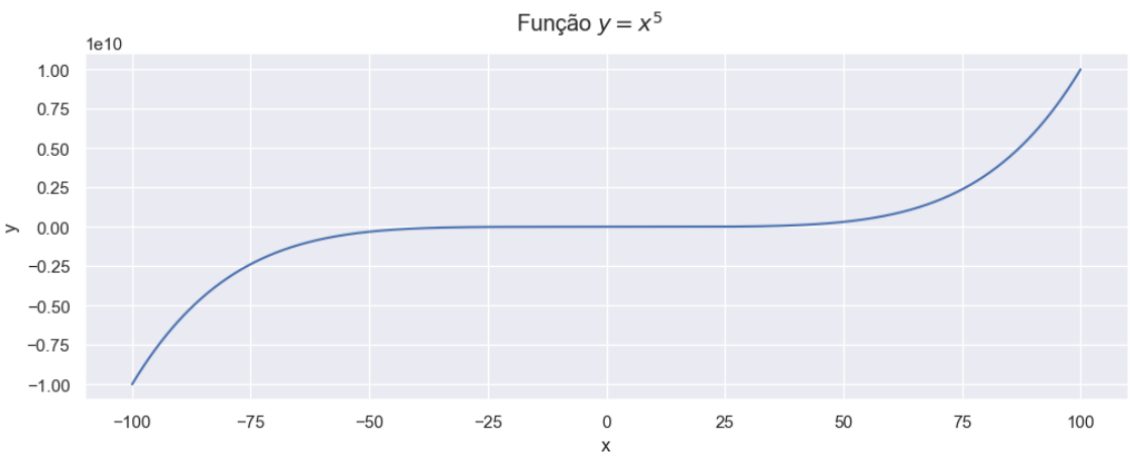
14. Cálculo de coeficientes de correlação

Durante esse capítulo, vamos explorar a ideia de correlação, analisando casos em que a relação entre variáveis não é estritamente linear, e comparar os coeficientes de correlação de **Pearson** e **Spearman**, entendendo quando e como utilizar cada um.

Correlação e Linearidade

Vamos começar com um exemplo clássico: plotar a função y=x5y = x^5y=x5. Apesar da função ser crescente, a relação entre xxx e yyy não é linear.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr

sns.set()
sns.set_context('notebook')

x = np.linspace(-100, 100, 1000)
y = x**5

corr, _ = pearsonr(x, y)
print(f'Correlação entre X e X^5: {corr:.3f}')

fig, ax = plt.subplots(figsize=(12, 4))
sns.lineplot(ax=ax, x=x, y=y)
ax.set_xlabel('x')
ax.set_ylabel('y')
fig.suptitle('Função $y = x^5$')
plt.show()
```



Apesar de os valores de x e y estarem ordenados — ou seja, à medida que x cresce, y também cresce —

a **correlação de Pearson** não é perfeita, pois essa métrica mede a **proporcionalidade linear**. Como x5x^5x5 cresce de forma acelerada, a proporcionalidade se perde, e o coeficiente de correlação é inferior a 1.

Correlação de Spearman

Se quisermos avaliar apenas se as variáveis crescem (ou decrescem) juntas, sem nos preocupar com a proporcionalidade, usamos a **correlação de Spearman**.

```
from scipy.stats import spearmanr

pearson_corr, _ = pearsonr(x, y)
spearman_corr, _ = spearmanr(x, y)

print(f'Correlação de Pearson entre X e X^5: {pearson_corr:.3f}')
print(f'Correlação de Spearman entre X e X^5: {spearman_corr:.3f}')
```

Resultado esperado:

- Pearson ≈ 0.82
- Spearman = 1.0

A correlação de Spearman é 1.0 porque, ao ordenar os valores de x e y, a ordem relativa é mantida — ou seja, é uma relação **monotônica perfeita**.

Visualizando Correlações em um DataFrame

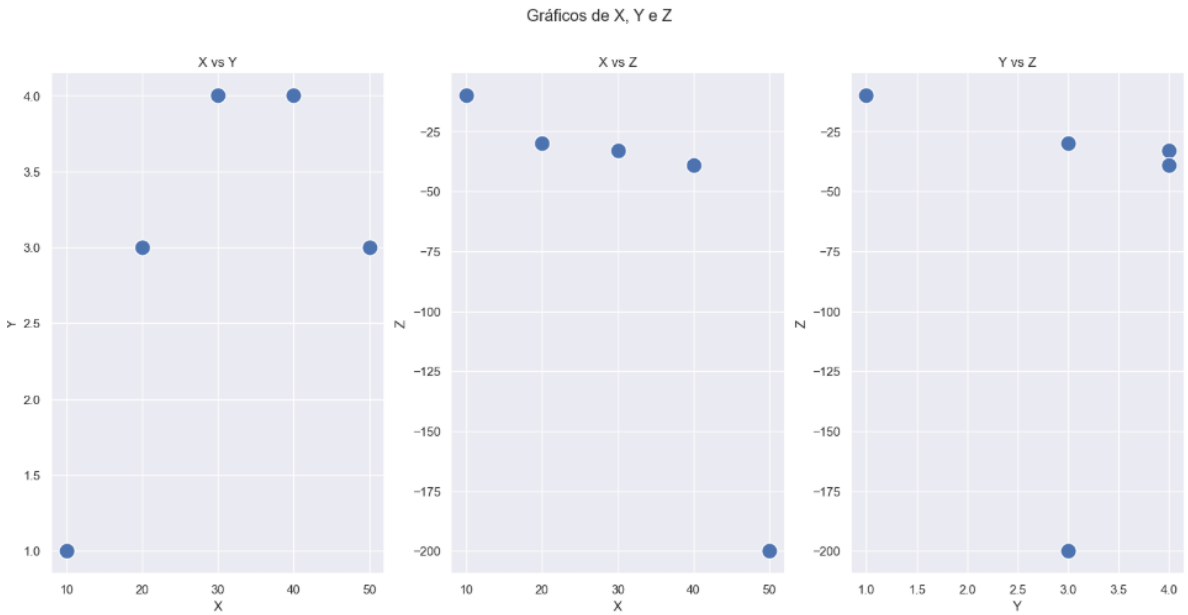
Vamos montar um pequeno DataFrame para explorar visualmente as diferenças entre as duas correlações:

```
import pandas as pd

df = pd.DataFrame({
    'X': [10, 20, 30, 40, 50],
    'Y': [1, 3, 4, 4, 3],
    'Z': [-10, -30, -33, -39, -200],
})
```

Gráficos de Dispersão

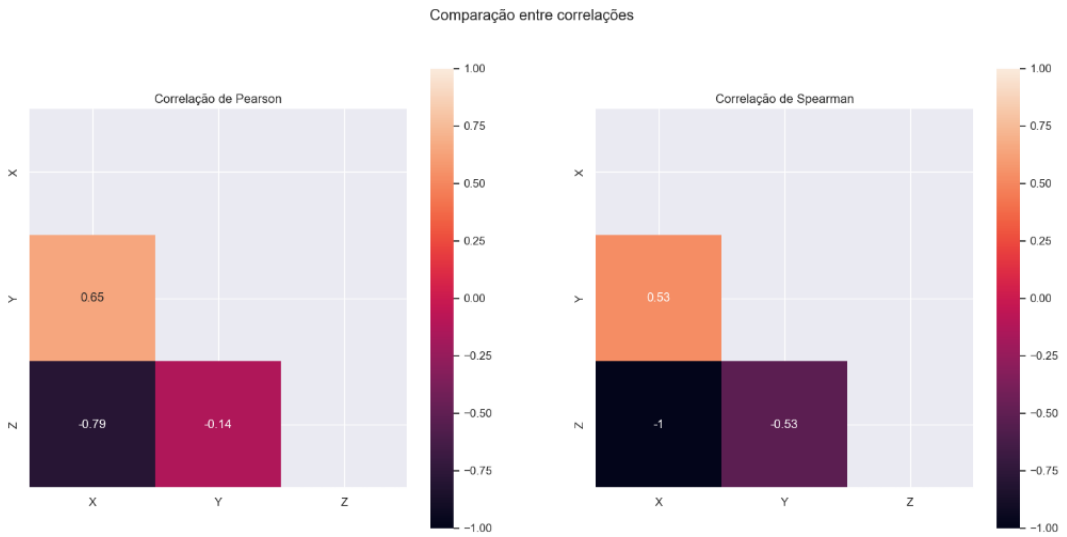
```
fig, (ax1, ax2, ax3) = plt.subplots(figsize=(18, 8), ncols=3)

sns.scatterplot(data=df, ax=ax1, x='X', y='Y', s=200)
ax1.set_title('X vs Y')
sns.scatterplot(data=df, ax=ax2, x='X', y='Z', s=200)
```

```
ax2.set_title('X vs Z')
sns.scatterplot(data=df, ax=ax3, x='Y', y='Z', s=200)
ax3.set_title('Y vs Z')

fig.suptitle('Gráficos de X, Y e Z')
plt.show()
```



Comparando Pearson e Spearman

```
mask = np.triu(df.corr())

fig, (ax1, ax2) = plt.subplots(figsize=(18, 8), ncols=2)

sns.heatmap(data=df.corr('pearson'), ax=ax1, annot=True, cbar=True, square=True,
            cmap='rocket', vmax=1.0, vmin=-1.0, mask=mask)
ax1.set_title('Correlação de Pearson')

sns.heatmap(data=df.corr('spearman'), ax=ax2, annot=True, cbar=True, square=True,
            cmap='rocket', vmax=1.0, vmin=-1.0, mask=mask)
ax2.set_title('Correlação de Spearman')

fig.suptitle('Comparação entre correlações')
plt.show()
```



É possível observar que:

- A correlação **Pearson** entre Y e Z é próxima de 0 (≈ -0.14), pois não há proporcionalidade linear.
- Já a correlação **Spearman** (≈ -0.53) detecta uma ordem inversa, o que reflete uma correlação **monotonicamente negativa** mais forte.

Quando Usar Pearson ou Spearman?

Cenário	Correlação mais adequada
Você quer entender **quanto** uma variável muda proporcionalmente com outra (ex: "a cada 10 cliques, aumento 2 vendas")	**Pearson**
Você quer entender apenas **a ordem dos valores**, ou seja, se ao aumentar uma variável, a outra também tende a aumentar (ou diminuir), sem se preocupar com proporções	**Spearman**

Em termos estatísticos:

- **Pearson**: correlação **paramétrica**, depende dos valores absolutos.
- **Spearman**: correlação **não-paramétrica**, baseada apenas no **ranking** dos dados.

15. Modelos de regressão linear

Para finalizar nosso curso de Análise Exploratória de Dados (EDA), vamos abordar um dos conceitos mais conhecidos da estatística: a **regressão linear**. Não vamos nos aprofundar na matemática por trás dela, mas sim mostrar como aplicar esse modelo de forma prática e útil durante a análise exploratória de dados.

O que é regressão linear?

A regressão linear é uma maneira de encontrar uma **reta** que melhor representa a relação entre duas variáveis numéricas. Ela está diretamente relacionada ao conceito de correlação, que vimos anteriormente: se duas variáveis possuem correlação (positiva ou negativa), é provável que exista uma reta que as conecte de maneira significativa.

A equação da regressão linear é familiar para quem já estudou funções matemáticas:

$$y = ax + b$$

- a representa a **inclinação** da reta (ou "slope");
- b é o **intercepto**, ou seja, o ponto em que a reta cruza o eixo y.

A partir dessa equação, conseguimos modelar como uma variável depende da outra. No contexto de dados, isso pode significar, por exemplo, estimar o salário de uma pessoa com base em seus anos de experiência.

Aplicando a regressão linear com `scipy`

Vamos aplicar a regressão linear utilizando os dados da África do Sul do nosso dataset. Selecionamos as colunas de anos de experiência (`WorkExp`) e salário anual convertido (`ConvertedCompYearly`), removendo os valores ausentes.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import linregress

df = pd.read_csv('materiais de aula/survey_results_public.csv', index_col='ResponseId')

df_south_africa = df.loc[df['Country'] == 'South Africa'][['WorkExp',
↪  'ConvertedCompYearly']].dropna()
x = df_south_africa['WorkExp']
```

```
y = df_south_africa['ConvertedCompYearly']

regression = linregress(x, y)
```

A função `linregress` retorna os coeficientes `slope`, `intercept`, e também o valor `rvalue`, que representa o **coeficiente de correlação de Pearson**.

```
print(f'Reta de regressão linear: y = {regression.slope:.2f}x + {regression.intercept:.2f}')
print(f'Coeficiente de correlação (Pearson): {regression.rvalue}')
```

Um valor de correlação próximo de 1 ou -1 indica uma forte relação linear. Valores próximos de 0 sugerem uma fraca (ou inexistente) relação linear.

Visualizando a regressão linear

Podemos visualizar os dados e a reta estimada com `matplotlib` e `seaborn`:

```
x_reg = np.linspace(0, 50, 100)
y_reg = x_reg * regression.slope + regression.intercept

fig, ax = plt.subplots(figsize=(12, 4))

sns.scatterplot(data=df_south_africa, ax=ax, x='WorkExp', y='ConvertedCompYearly')
sns.lineplot(ax=ax, x=x_reg, y=y_reg, color='#efab02')

fig.suptitle(f'Anos de experiência X salário - África do Sul (corr: {regression.rvalue:.2f})')

plt.show()
```

Esse gráfico nos ajuda a entender visualmente como os dados se alinham (ou não) com a reta de regressão. Notamos a presença de **outliers**, que estão bem distantes da tendência geral e podem influenciar negativamente os resultados.

Filtrando os outliers

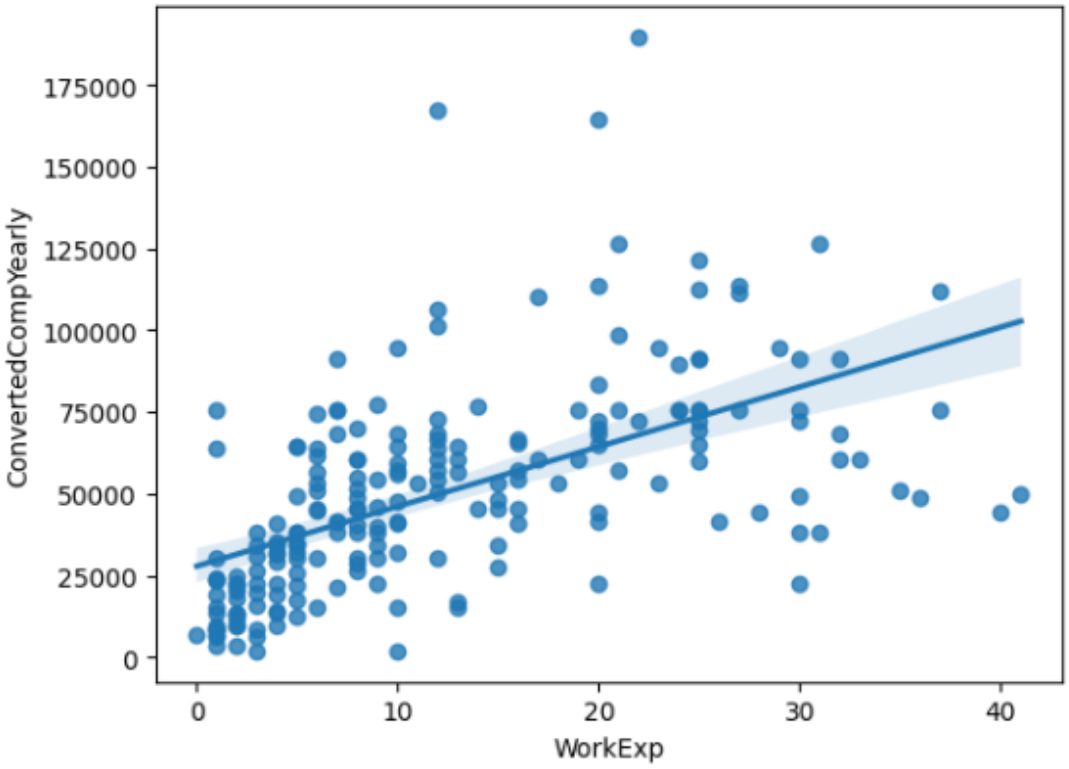
Vamos limitar os dados a salários abaixo de 200 mil para reduzir a distorção causada pelos outliers:

```
df_south_africa_filtered = df_south_africa[df_south_africa['ConvertedCompYearly'] < 200_000]

x = df_south_africa_filtered['WorkExp']
y = df_south_africa_filtered['ConvertedCompYearly']
regression = linregress(x, y)
```

Usando o `seaborn.regplot`

O Seaborn oferece uma forma ainda mais prática de fazer isso:

```
sns.regplot(data=df_south_africa_filtered, x='WorkExp', y='ConvertedCompYearly')
```



Essa função plota a reta de regressão automaticamente e ainda mostra um **intervalo de confiança** em volta dela, indicado por uma faixa sombreada. Quanto menor essa faixa, maior a confiança que temos no modelo.

Outras possibilidades com `regplot`

O `regplot` também permite ajustes interessantes, como:

- **Regressão polinomial** (curvas ao invés de retas):

```
sns.regplot(data=df_south_africa_filtered, x='WorkExp', y='ConvertedCompYearly', order=2)
```

- **Transformação logarítmica** dos dados (ajuda quando os dados têm grande dispersão):

```
sns.regplot(data=df_south_africa_filtered + 0.01, x='WorkExp', y='ConvertedCompYearly',
↪  logx=True)
```

- **Regressão logística** (não abordada neste curso, mas útil para variáveis categóricas):

```
sns.regplot(data=df_south_africa_filtered + 0.01, x='WorkExp', y='ConvertedCompYearly',
↪  logistic=True)
```

Considerações finais

A regressão linear é uma ferramenta poderosa de modelagem e visualização de padrões. Seja para inferir tendências, fazer predições simples ou apenas explorar relações entre variáveis, ela é uma aliada valiosa em EDA.

Lembre-se de que:

- Nem sempre uma regressão faz sentido — é importante observar os dados;
- Outliers podem distorcer muito os resultados;
- Métodos visuais como os do `seaborn` ajudam a detectar rapidamente boas (ou más) regressões.